

МИНОБРНАУКИ РОССИИ  
Федеральное государственное бюджетное образовательное  
учреждение высшего образования  
«Ярославский государственный университет имени П. Г. Демидова»

Кафедра дискретного анализа

Сдано на кафедру

«\_\_\_\_\_» \_\_\_\_\_ 2026 г.

Заведующий кафедрой,

д. ф.-м. н., доцент

\_\_\_\_\_ А. В. Николаев

Выпускная квалификационная работа

**Разработка игрового приложения  
«Коридор» с модулем стратегического поведения  
соперника**

по направлению

01.04.02 Прикладная математика и информатика

Научный руководитель

д. ф.-м. н., доцент

\_\_\_\_\_ А. В. Николаев

«\_\_\_\_\_» \_\_\_\_\_ 2026 г.

Студент группы ИВТ-21МО

\_\_\_\_\_ А. О. Карпунин

«\_\_\_\_\_» \_\_\_\_\_ 2026 г.

Ярославль, 2026

# Реферат

Объем 79 с., 5 гл., 19 рис., 16 табл., 10 источников, 4 прил.

Ключевые слова: **игры, Коридор, поиск кратчайшего пути, игровой искусственный интеллект, Минимакс, альфа-бета отсечение, Монте-Карло, функция оценки, обучение весов.**

Объект исследования — логическая игра «Коридор» и алгоритмы автоматического расчета лучшего хода в ней.

Цель работы — разработка игрового приложения, с помощью которого пользователь сможет сыграть против одного из четырёх программных соперников и провести их сравнение с помощью проведения турнира в качестве эксперимента.

Результат работы — приложение, которое реализует правила игры «Коридор», с возможностью игры против компьютера или наблюдения за игрой алгоритмов друг против друга.

# Содержание

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| <b>Введение</b>                                                       | <b>5</b>  |
| <b>1. Игра «Коридор»</b>                                              | <b>7</b>  |
| 1.1. Необходимые определения . . . . .                                | 7         |
| 1.2. Описание игры . . . . .                                          | 8         |
| <b>2. Модуль стратегического поведения соперника</b>                  | <b>9</b>  |
| 2.1. Оценка сложности игрового дерева . . . . .                       | 9         |
| <b>3. Алгоритмы</b>                                                   | <b>11</b> |
| 3.1. Общие функции и структура алгоритмов . . . . .                   | 11        |
| 3.2. Случайный алгоритм . . . . .                                     | 18        |
| 3.3. Минимакс . . . . .                                               | 18        |
| 3.4. Альфа-бета отсечения . . . . .                                   | 22        |
| 3.5. Монте-Карло . . . . .                                            | 24        |
| 3.5.1. Выбор . . . . .                                                | 24        |
| 3.5.2. Расширение . . . . .                                           | 25        |
| 3.5.3. Симуляция . . . . .                                            | 25        |
| 3.5.4. Обратное распространение . . . . .                             | 26        |
| 3.5.5. Выбор лучшего хода . . . . .                                   | 26        |
| 3.6. Сводные параметры сложности . . . . .                            | 27        |
| <b>4. Сравнение алгоритмов</b>                                        | <b>28</b> |
| 4.1. Методика сравнения . . . . .                                     | 28        |
| 4.2. Собираемые метрики . . . . .                                     | 28        |
| 4.3. План эксперимента . . . . .                                      | 28        |
| 4.4. Условия проведения эксперимента . . . . .                        | 29        |
| 4.5. Таблицы результатов . . . . .                                    | 29        |
| 4.5.1. Результаты эксперимента на большом поле . . . . .              | 29        |
| 4.5.2. Результаты эксперимента на малых полях . . . . .               | 30        |
| 4.5.3. Зависимость вычислительной стоимости от размера поля . . . . . | 32        |
| 4.6. Сравнение весов функции оценки . . . . .                         | 33        |
| 4.7. Анализ результатов . . . . .                                     | 34        |
| <b>5. Игровое приложение</b>                                          | <b>35</b> |
| 5.1. Техническая реализация . . . . .                                 | 35        |
| 5.2. Взаимодействие с приложением . . . . .                           | 37        |

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>Заключение</b>                                             | <b>41</b> |
| <b>Список литературы</b>                                      | <b>42</b> |
| <b>Приложение А. Реализация алгоритма MiniMax</b>             | <b>43</b> |
| <b>Приложение Б. Реализация алгоритма AlphaBeta</b>           | <b>50</b> |
| <b>Приложение В. Реализация алгоритма Monte Carlo</b>         | <b>63</b> |
| <b>Приложение Г. Реализация обучения весов функции оценки</b> | <b>74</b> |

## Введение

«Коридор» — настольная логическая игра для двух или четырёх игроков, которая была выпущена в 1997 году компанией Gigamic. Ее создатель, Мирко Маркези, разработал ее, взяв за основу ранее выпущенную игру «Блокада». «Коридор» получила заметное распространение среди настольных логических игр и вошла в подборку Games 100 журнала *Games* [1].

Цель у игрока такова — необходимо довести свою фишку до противоположной стороны игрового поля быстрее, чем это сделает соперник. На каждом ходу игрок выбирает одно из следующих действий: продвинуть свою фишку к противоположной стороне поля или поставить стену, которая усложнит маршрут соперника. Количество стен ограничено, а правила игры запрещают полностью блокировать путь любого игрока к его целевой стороне поля.

Задача создания программного соперника для логической игры обычно рассматривается через модель игры с полной информацией, в которой возможные продолжения партии образуют дерево состояний. Для таких игр применяются алгоритмы перебора дерева, в частности Минимакс и альфа-бета отсечения, позволяющие учитывать ответные действия соперника [3]. Общие свойства математических игр и анализа позиций также рассматриваются в работах по комбинаторной теории игр [6].

Для игры «Коридор» уже разрабатывались программные соперники. Так, в работе V. Massagué Respal, J.A. Brown и H. Aslam, описывается использование алгоритма Монте Карло и его сравнение с уже существующими подходами [7]. В исследовательском отчёте Quoridor-AI сравниваются несколько подходов к построению агента для этой игры, включая алгоритмы Minimax, Deep Q-Network и генетический алгоритм [8].

В данной работе акцент сделан не на разработке одного отдельного агента, а на создании игрового приложения, в котором несколько алгоритмов могут быть использованы как программные соперники, противники в турнире друг с другом, помощники в рамках механизма получения подсказок к ходу и т.д.

Целью выпускной квалификационной работы является разработка игрового приложения «Коридор», включающего в себя реализацию правил игры, визуализацию игрового поля и модуль стратегического поведения, объединяющий четыре алгоритма программного соперника.

Для реализации цели работы были поставлены следующие задачи:

1. изучить правила игры «Коридор»;
2. реализовать алгоритм проверки корректности перемещения фишки;

3. реализовать алгоритм проверки корректности установки перегородки с условием достижимости целевых сторон поля;
4. реализовать алгоритм поиска кратчайшего пути до целевой стороны поля на основе обхода графа в ширину;
5. разработать функцию оценки позиции игрока с настраиваемыми весами и механизмом их коррекции по результатам партий;
6. реализовать четыре алгоритма программного соперника: случайный выбор хода, Минимакс, альфа-бета отсечение и Монте-Карло;
7. провести сравнение алгоритмов в серии автоматических партий;
8. разработать интерфейс игрового приложения.

# 1. Игра «Коридор»

## 1.1. Необходимые определения

В формулировке правил игры и последующем описании алгоритмов встречается несколько понятий, которые нуждаются в пояснении.

*Игровым полем* называется квадратная доска размером  $n \times n$  клеток. В стандартном варианте игры  $n = 9$ , однако приложение также поддерживает поля  $7 \times 7$  и  $11 \times 11$ . Каждая из клеток нумеруется парой индексов  $(i, j)$ , где  $0 \leq i, j \leq n - 1$ .

Пример игрового поля из разработанного приложения приведён на рис. 1. На нём показаны клетки поля, фишки игроков и уже установленные стены.

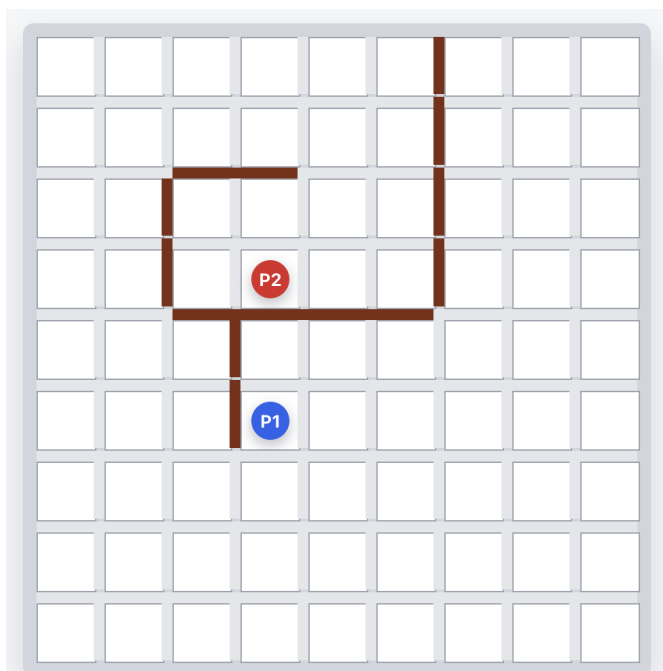


Рис. 1 — Игровое поле приложения «Коридор»

*Фишкой* называется объект, который занимает одну клетку игрового поля и может быть передвинут на одну из смежных клеток.

*Стеной* называется элемент, который занимает пространство между клетками. Такая перегородка имеет длину две клетки и может быть расположена горизонтально, блокируя вертикальный проход между двумя парами клеток, или вертикально. Каждому игроку в начале игры выделено ограниченное количество стен: 8 для поля  $7 \times 7$ , 10 для  $9 \times 9$ , 12 для  $11 \times 11$ .

*Целевой линией* игрока называется строка игрового поля, которая находится на противоположной стороне от его стартовой позиции. В начале партии игроки расположены на центрах противоположных сторон поля.

## 1.2. Описание игры

В начале партии фишки игроков расставляются на противоположных сторонах поля: первый игрок — в центре нижней строки, второй — в центре верхней. Они ходят по очереди, первенство определяется случайно. Цель каждого игрока — первым достичь любой клетки своей целевой линии.

За один ход игрок может выполнить только одно из двух действий [2]:

1. Перемещение по игровому полю. Фишка игрока передвигается на смежную клетку, если такой маневр не заблокирован стеной. Если соседняя клетка занята фишкой соперника и за ней нет ни перегородки, ни границы поля, фишка игрока может перепрыгнуть через соперника. Если прямой прыжок невозможен, допускается прыжок на клетку слева или справа от той, куда изначально предполагался прыжок. Варианты перемещения показаны на рис. 2.
2. Установка стены. Она допускается, если у игрока есть хотя бы одна перегородка в его запасе, она не пересекается с уже установленными стенами на игровом поле и после её установки у каждого из игроков остается хотя бы один маршрут к своей целевой линии. Пример установки стены приведён на рис. 3.

Игра завершается, когда фишка одного из игроков достигает любой клетки целевой линии.

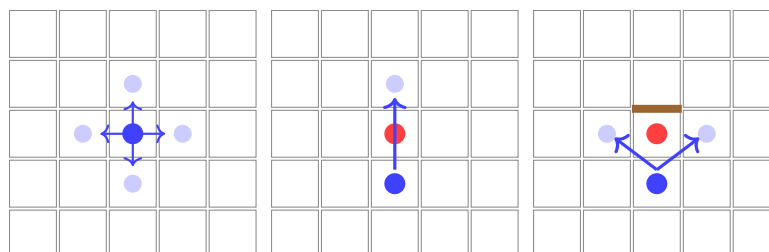


Рис. 2 — Варианты перемещения фишки

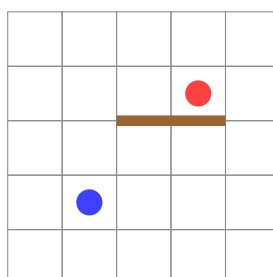


Рис. 3 — Пример установки стены



## 2. Модуль стратегического поведения соперника

Модуль стратегического поведения — это программный компонент, который отвечает за выбор хода соперника. Основываясь на полученных параметрах, а именно текущем состоянии игрового поля, количестве оставшихся стен у игроков, модуль возвращает лучший по собственной оценке ход.

В процессе разработки к модулю стратегического поведения соперника было выдвинуто несколько требований:

1. Выбранный алгоритмом ход не должен противоречить правилам игры.
2. Алгоритм должен завершаться за конечное время. Все реализованные в приложении алгоритмы работают с ограничением по времени: каждый из них получает лимит в миллисекундах и прекращает поиск по его истечении. Этот устанавливаемый лимит зависит от сложности игры и выбранного размера игрового поля.
3. Все алгоритмы должны реализовывать единый программный интерфейс `Algorithm` с методом получения лучшего хода. Благодаря этому переключение алгоритмов или добавление новых не требует вносить какие-либо изменения в остальные компоненты игрового приложения.
4. Каждый из алгоритмов должен поддерживать три уровня сложности. Выбранный уровень сложности влияет не только на ограничение по времени, но и на глубину поиска или число итераций алгоритма.

В рамках данной работы в модуле поведения соперника реализовано четыре алгоритма, каждый из которых может быть выбран в качестве оппонента. Случайный алгоритм равновероятно выбирает один из множества допустимых ходов. Минимакс перебирает дерево игровых состояний на ограниченную глубину без каких-либо отсечений. Альфа-бета расширяет минимакс, добавляя в изначальный алгоритм отсечения безперспективных ветвей дерева. Метод Монте-Карло основан на оценке игровых позиции через многократные случайные симуляции игры из каждой из них.

### 2.1. Оценка сложности игрового дерева

Основная сложность разработки программного соперника для игры «Коридор» связана с большим числом допустимых продолжений партии. В каждый ход игрок может не только передвинуть фишку, но и поставить стену. Для поля  $n \times n$  максимальное число позиций одной стены без учёта коллизий равно  $2(n - 1)^2$ .

Поэтому даже грубая оценка коэффициента ветвления имеет вид

$$b(n) \approx 2(n - 1)^2 + 4,$$

где 4 соответствует возможным перемещениям фишки. На практике число ходов меньше из-за занятых клеток, уже установленных стен и правил достижимости целевых линий, однако зависимость от размера поля остаётся квадратичной. При увеличении глубины поиска число просматриваемых узлов растёт уже как  $b(n)^d$ , где  $d$  — глубина дерева в полуходах.

Таблица 2.1

**Оценка роста игрового дерева при изменении размера поля**

| Поле           | Позиций стен | Оценка $b(n)$ | Узлов при $d = 4$ | Узлов при $d = 6$    |
|----------------|--------------|---------------|-------------------|----------------------|
| $7 \times 7$   | 72           | 76            | $3,34 \cdot 10^7$ | $1,93 \cdot 10^{11}$ |
| $9 \times 9$   | 128          | 132           | $3,04 \cdot 10^8$ | $5,29 \cdot 10^{12}$ |
| $11 \times 11$ | 200          | 204           | $1,73 \cdot 10^9$ | $7,21 \cdot 10^{13}$ |

Для полного дерева игры  $d$  соответствует не глубине просмотра одного хода, а длине партии до терминального состояния. При увеличении поля растёт не только коэффициент ветвления  $b(n)$ , но и потенциальная глубина полного дерева  $d(n)$ . В результате сложность полного перебора можно оценивать как

$$O(b(n)^{d(n)}).$$

Глубина игрового дерева зависит от длительности партии. Минимально партия требует порядка  $O(n)$  полуходов, однако наличие стен увеличивает длину маршрутов. Поскольку число стен у игроков масштабируется вместе с размером поля, практическую верхнюю оценку глубины партии можно рассматривать как  $O(n^2)$  (в ходе эксперимента было выяснено, что средняя длина партии для поля размером  $9 \times 9$  — 90 ходов, для поля  $7 \times 7$  — 50, а для  $11 \times 11$  — 110 ходов). Тогда общая оценка сложности дерева равна

$$T(n) = O(b(n)^{d(n)}) = O((n^2)^{n^2}).$$

Для стандартного поля  $9 \times 9$  приводится оценка сложности дерева игры порядка  $10^{162}$  [7]. Именно поэтому реализованные алгоритмы используют ограничение глубины, временной лимит, сортировку ходов и эвристическую функцию оценки позиции, чтобы не заниматься полным перебором всего игрового дерева.

## 3. Алгоритмы

### 3.1. Общие функции и структура алгоритмов

Каждый из разработанных алгоритмов реализуют единый программный интерфейс `Algorithm`, содержащий как абстрактные методы, тело которых определяется выбранным алгоритмом самостоятельно, так и конкретные реализации общих для всех алгоритмов вспомогательных функций. С помощью данного интерфейса получилось исключить дублирование кода и упростить добавление нового алгоритма в приложение.

#### Генерация допустимых ходов

Метод `getMoves(board, playerId, wallsLeft)`, который применяется всеми алгоритмами в модуле стратегического поведения соперника, формирует полное множество допустимых ходов из текущего состояния из двух частей.

Первая часть — допустимые перемещения фишки, получаемые с помощью метода `board.getAvailableMoves(pos)`, который поочередно проверяет все направления перемещения с текущей клетки, а также обрабатывает прыжки через соперника, если такие возможны.

Вторая часть — допустимые варианты установки стен. Алгоритм формирует множество «кандидатных» клеток, ограничивая полный перебор стратегически значимой областью поля. Кандидатами являются клетки, которые расположены около кратчайшего пути соперника к его целевой строке поля. Такой выбор связан с тем, что стена полезна прежде всего тогда, когда она меняет текущий лучший маршрут соперника. Если перегородка ставится далеко от этого маршрута, то с высокой вероятностью она не увеличивает длину пути до цели и только расширяет перебор, из-за чего увеличивается время, необходимое для выбора хода. Для таких клеток проверяются два возможных направления стены. Перегородка включается в список кандидатов, если она не имеет коллизий с уже расставленными перегородками на поле, и не отрезает ни одного из игроков от его целевой линии. Помимо прочего проверяется выполнение хотя бы одного условия:

1. стена увеличивает кратчайший путь соперника минимум на 2 хода,
2. собственный путь удлиняется меньше, чем путь соперника.

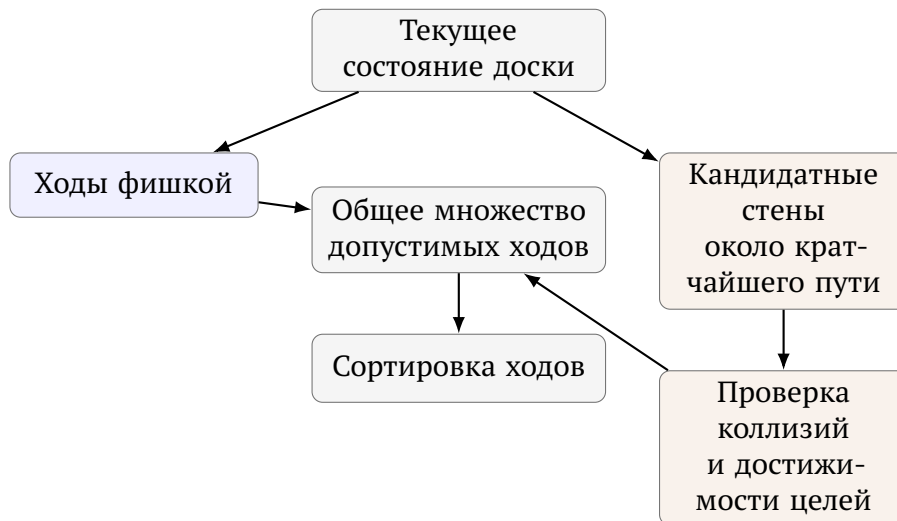
Количество рассматриваемых стен можно оценить через длину найденного маршрута. Если длина кратчайшего пути соперника равна  $l$ , то алгоритм анализирует не всё поле, а не более  $l$  «кандидатных» клеток или переходов, связанных с этим маршрутом. В начале партии для поля со стороной  $n$   $l = n - 1$ . Для каждой

«кандидатной» клетки проверяется ограниченное число локальных постановок стены, которые могут перекрыть движение по текущему маршруту или заставить соперника искать обход. Около одной клетки можно поставить не более восьми вариантов стены, то число таких вариантов можно грубо оценить как  $8l$ . Тогда на старте партии, с учётом нескольких ходов фишкой, ширина множества кандидатов имеет оценку

$$b(n) \leq 8(n - 1) + 3,$$

где 3 соответствует основным перемещениям фишки в начальной позиции. Для поля  $7 \times 7$  такая оценка даёт не более 51 кандидатов, для  $9 \times 9$  — не более 67, для  $11 \times 11$  — не более 83. В реальных позициях это число дополнительно уменьшается из-за границ поля, уже установленных стен и проверки сохранения пути для обоих игроков.

Итоговый список возможных стен сортируется по убыванию значения функции `wallImpact`. Схема формирования множества допустимых ходов приведена на рис. 4.



**Рис. 4** — Формирование множества допустимых ходов

### Функция `wallImpact`

Функция `wallImpact(board, move, playerId, size)` даёт установке стены оценку ее стратегической ценности. После применения хода на копии поля она вычисляет изменения длин путей участников игры до их целевых строк:

$$\text{wallImpact} = (d_p^{\text{after}} - d_p^{\text{before}}) \cdot 100 - \max(0, d_a^{\text{after}} - d_a^{\text{before}}) \cdot 45,$$

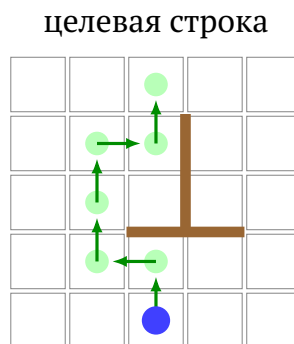
то есть каждый шаг, увеличивающий путь соперника, оценивается в 100 единиц, а каждый шаг, который приводит к увеличению собственного пути, штрафуются

45 очками. Стена в игровом приложении считается стратегически ценной, если  $wallImpact > 0$ .

### Поиск кратчайшего пути

В основе метода `calculateShortestPath(board, start, targetRow)` лежит обход графа в ширину от текущей позиции фишки на игровом поле до ближайшей клетки целевой строки [4].

Метод `getShortestPathCells` дополняет предыдущий восстановлением кратчайшего пути, поскольку при поиске такового хранит в истории посещенные клетки. Как уже описывалось ранее, он используется для поиска «кандидатных» клеток, рядом с которыми можно было бы установить стену. Принцип работы поиска кратчайшего пути на графе поля показан на рис. 5.



**Рис. 5** — Поиск кратчайшего пути с учётом установленных стен

### Эндшпильные правила

С метода `findEndgameMove` начинается любой поиска лучшего хода алгоритма. В случае, если с текущей клетки поля фишку игрока можно передвинуть прямо его на целевую линию, этот ход немедленно возвращается без перебора других вариантов. Если же такого «победного» хода нет, но соперник находится не более чем в двух ходах от победы, а лимит стен у алгоритма не исчерпан — ищется стена с максимальным значением `wallImpact`, чтобы отсрочить победу игрока.

### Предпочтение направления движения

С помощью метода `movementPreference` к оценке хода добавляется бонус или штраф в зависимости от направления перемещения и истории предыдущих ходов:

- ход в сторону целевой линии: +45 очков;
- ход назад: −120 очков;
- уменьшение длины пути до целевой линии на  $\Delta$  клеток:  $+35 \cdot \Delta$  очков;

- перемещение на позицию, которая встречалась в последних двух ходах: —500 очков;
- повтор более давнего перемещения: —180 очков.

Вся история перемещений хранится для каждого игрока отдельно и получается алгоритмом путем вызова `getRecentPositions`.

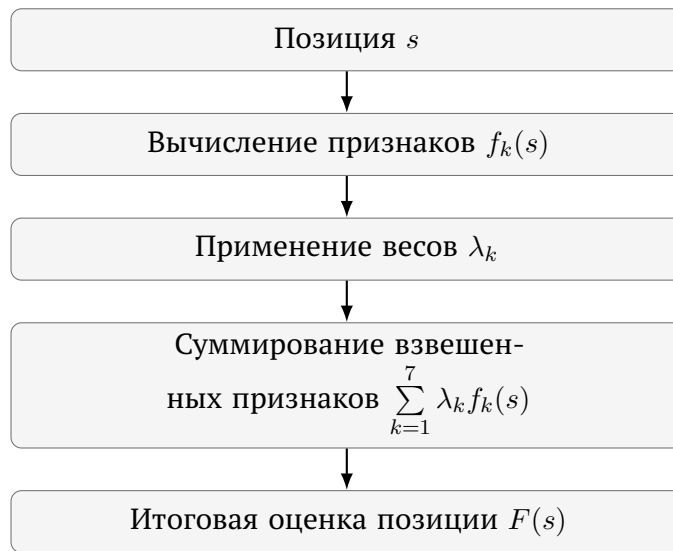
Такие значения применяются в качестве локальной поправки найденной оценки хода. Самый большой штраф получает возврат фишки алгоритма на недавно посещённую клетку. Он необходим для борьбы с «зацикливанием» ходов, когда алгоритм несколько ходов подряд перемещает фишку между двумя соседними клетками. За ход назад алгоритм получает меньший штраф, потому что в игре «Коридор» он может быть оправдан обходом стены, однако алгоритм не должен выбирать его без заметной причины. Бонус за движение к цели или уменьшение кратчайшего пути специально выбран ниже, чем штрафы за «зацикливание» хода. Они подталкивают алгоритм двигать фишку к целевой линии, но не перекрывают основные факторы функции оценки, связанные с длиной путей, стенами и победными позициями.

### Функция оценки позиции

Функция оценки `evaluate()` является главным компонентом всех реализованных в игровом приложении алгоритмов. Благодаря ей каждому состоянию игры сопоставляется числовое значение, которое описывает его стратегическую выгоду для программного соперника. Она необходима, поскольку практически невозможно совершить полный перебор дерева игры за ограниченное время [5]. Оценка вычисляется как взвешенная сумма нескольких параметров:

$$F(s) = \sum_{k=1}^7 \lambda_k \cdot f_k(s),$$

где  $\lambda_k$  — веса, а  $f_k(s)$  — значения параметров. Общая идея вычисления оценки показана на рис. 6. Параметры и их описание перечислены в таблице 3.1.



**Рис. 6** — Вычисление оценки игровой позиции

Таблица 3.1

**Параметры оценочной функции**

| $k$ | Признак $f_k(s)$                | Описание                                                         |
|-----|---------------------------------|------------------------------------------------------------------|
| 1   | $d_p - d_a$                     | Разность между длинами кратчайших путей алгоритма и пользователя |
| 2   | $\text{mob}_a$                  | Количество допустимых ходов фишкой алгоритма — его мобильность   |
| 3   | $-\text{mob}_p$                 | Отрицательное количество допустимых ходов пользователя           |
| 4   | $w_a - w_p$                     | Преимущество алгоритма по числу оставшихся стен                  |
| 5   | $\text{prog}_a - \text{prog}_p$ | Разность продвижений к целевым линиям алгоритма и пользователя   |
| 6   | $e(d_a)$                        | Бонус за близость алгоритма к победе над игроком                 |
| 7   | $-e(d_p)$                       | Штраф за близость пользователя к победе над алгоритмом           |

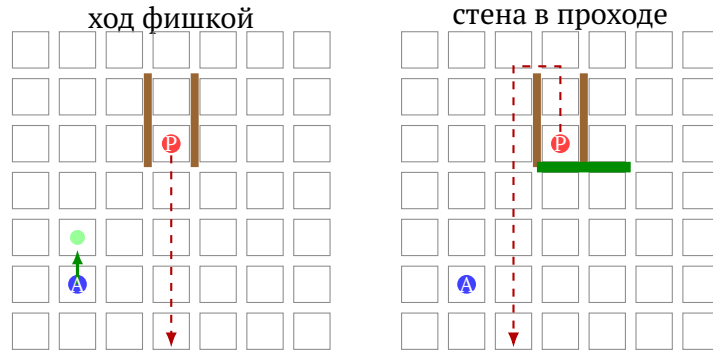
Продвижение  $\text{prog}_a$  — это количество строк игрового поля, которые прошла фишка алгоритма от стартовой позиции до текущей. Функция  $e(d)$  может принимать такие значения:

$$e(d) = \begin{cases} 10, & d \leq 1, \\ 4, & d = 2, \\ 1, & d = 3, \\ 0, & d > 3. \end{cases}$$

Веса  $\lambda_k$ , соответствующие описанным параметрам, по умолчанию заданы следующим образом (класс `EvaluationWeights`):

$$\lambda_1 = 65, \quad \lambda_2 = 4, \quad \lambda_3 = 3, \quad \lambda_4 = 18, \quad \lambda_5 = 6, \quad \lambda_6 = 165, \quad \lambda_7 = 175.$$

Пример использования функции оценки удобно рассмотреть на конкретной позиции. На рис. 7 показаны два варианта продолжения для синей фишки алгоритма: обычный ход фишкой вперёд и установка стены в узком проходе перед соперником. Красная фишка игрока находится в коридоре: слева и справа от неё уже стоят перегородки. Поэтому новая стена во втором варианте перекрывает движение вперёд и вынуждает игрока сначала отойти назад, а затем искать обходной путь. После применения каждого из этих вариантов для получившейся позиции вычисляется вектор признаков, а затем итоговая оценка по формуле  $F(s)$ .



**Рис. 7** — Два варианта хода, сравниваемые функцией оценки

Числовое сравнение этих вариантов приведено в таблице 3.2. Второй ход получает большую оценку, потому что сильнее увеличивает разницу между кратчайшими путями игроков, несмотря на расход одной стены.

Таблица 3.2

**Пример сравнения двух ходов с помощью функции оценки**

| Вариант           | $d_a$ | $d_p$ | $d_p - d_a$ | $\text{mob}_a$ | $-\text{mob}_p$ | $w_a - w_p$ | $\text{prog}_a - \text{prog}_p$ | $e(d_a)$ | $-e(d_p)$ | $F(s)$ |
|-------------------|-------|-------|-------------|----------------|-----------------|-------------|---------------------------------|----------|-----------|--------|
| Ход фишкой вперёд | 4     | 4     | 0           | 4              | -2              | 0           | 1                               | 0        | 0         | 16     |
| Установка стены   | 5     | 9     | 4           | 4              | -1              | -1          | 0                               | 0        | 0         | 255    |



Для первого варианта итоговая оценка равна

$$0 \cdot 65 + 4 \cdot 4 - 2 \cdot 3 + 0 \cdot 18 + 1 \cdot 6 = 16.$$

Для второго варианта:

$$4 \cdot 65 + 4 \cdot 4 - 1 \cdot 3 - 1 \cdot 18 = 255.$$

### Коррекция весов

Игровое приложение поддерживает механизм автоматической коррекции параметров функции оценки, анализируя результаты сыгранных партий (класс `EvaluationWeightsStore`). После завершения каждой из партий, в которых участвовал программный соперник, вычисляется оценка полезности ходов, которые предпринимал алгоритм.

В течение игры для каждого из ходов алгоритма формируется специальный объект `EvaluationLearningSample`, который хранит вектор параметров до хода и после него. После завершения партии для каждого образца вычисляется дельта параметров оценки  $\Delta f_k = f_k^{\text{after}} - f_k^{\text{before}}$  и суммируется в градиент с учётом знака:

$$g_k = \sum_{\text{ходы алгоритма}} \text{reward} \cdot \Delta f_k, \quad \text{reward} = \begin{cases} +1, & \text{алгоритм победил,} \\ -1, & \text{алгоритм проиграл.} \end{cases}$$

Если  $g_k > 0$ , увеличение веса  $k$ -го параметра чаще встречалось в ходах победителя, поэтому вес увеличивается. Если  $g_k < 0$ , этот признак чаще участвовал в принятии решений, приводящих к проигрышу, поэтому соответствующий ему вес уменьшается. Если  $g_k = 0$ , вес не изменяется. Для начала необходимо вычислить предварительное значение обновленного веса:

$$\lambda'_k = \lambda_k + \text{sign}(g_k) \cdot \Delta \lambda.$$

Затем оно проверяется согласно допустимому диапазону:

$$\lambda_k^{\text{new}} = \begin{cases} \lambda_k^{\min}, & \lambda'_k < \lambda_k^{\min}, \\ \lambda_k^{\max}, & \lambda'_k > \lambda_k^{\max}, \\ \lambda'_k, & \lambda_k^{\min} \leq \lambda'_k \leq \lambda_k^{\max}. \end{cases}$$

Здесь  $\Delta \lambda = 1$  — это шаг обучения, а  $\lambda_k^{\min}$  и  $\lambda_k^{\max}$  — минимально и максимально допустимые значения веса. Такие ограничения были наложены на веса параметров, чтобы после серии обучающих партий с алгоритмом отдельный параметр не стал чрезмерно большим, отрицательным или слишком малым и не начал

полностью подавлять остальные признаки функции оценки. Актуальные веса параметров регулярно сохраняются в файл `data/evaluation-weights.properties` и используются для функции оценки в следующем запуске приложения, что обеспечивает накопление опыта между сессиями. На момент написания работы механизм коррекции предпринял 15 обновлений весов на 1388 обучающих образцах. Текущие сохранённые веса таковы:

$$\lambda_1 = 120, \quad \lambda_2 = 7, \quad \lambda_3 = 12, \quad \lambda_4 = 8, \quad \lambda_5 = 25, \quad \lambda_6 = 186, \quad \lambda_7 = 170.$$

Сравнение изначально выбранных и обновленных весов показывает, что механизм увеличил приоритет разности путей ( $\lambda_1 : 65 \rightarrow 120$ ), штрафа за количество возможных ходов соперника ( $\lambda_3 : 3 \rightarrow 12$ ), продвижения к целевой строке игрового поля ( $\lambda_5 : 6 \rightarrow 25$ ) и собственного преимущества ( $\lambda_6 : 165 \rightarrow 186$ ), одновременно с этим уменьшив влияние преимущества по оставшимся стенам ( $\lambda_4 : 18 \rightarrow 8$ ) на общую оценку состояния игры.

### 3.2. Случайный алгоритм

`RandomAlgorithm` является простейшим из возможных программных соперников. При каждом вызове метода `getMove` алгоритм первым делом применяет эндшпильное правило: если можно немедленно выиграть самому или срочно заблокировать путь для соперника, он делает это независимо от уровня сложности. Если такое правило не сработало, поведение определяется уровнем сложности.

На уровне `EASY` алгоритм с равной вероятностью выбирает ход из полного множества допустимых ходов.

На уровне сложности `MEDIUM` ходы сортируются по убыванию значения оценки хода, и случайно выбирается один из первой списка ходов, ограниченного первой его половиной.

На уровне `HARD` случайно выбирается уже один из трёх лучших ходов с помощью применения той же оценки.

Общая схема работы алгоритма приведена в алгоритме 1.

### 3.3. Минимакс

В `MinimaxAlgorithm` во время хода алгоритма выбирается ход с максимальной оценкой (максимизирующий узел), на ходах пользователя предполагается выбор хода с минимальной оценкой (минимизирующий узел).

Алгоритм итеративно увеличивает глубину анализа хода: последовательно запускает поиск с глубиной  $d = 1, 2, \dots, d_{\max}$  до истечения лимита времени. На каждом этапе цикла в специальное поле сохраняется лучший выбранный ход, зна-

---

**Алгоритм 1:** Случайный алгоритм

---

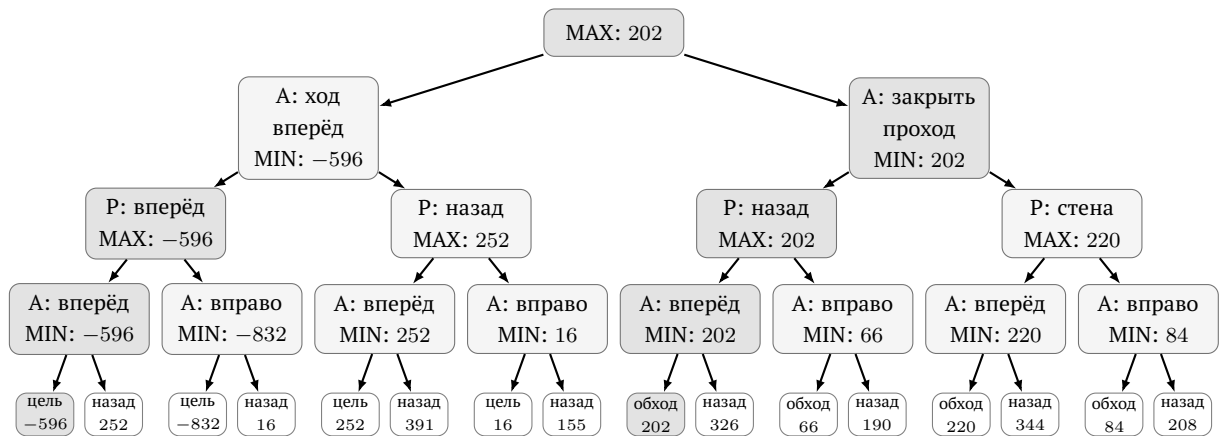
```
1 эндишпильный ход  $\leftarrow$  findEndgameMove( $s$ );
2 if эндишпильный ход найден then
3   | return эндишпильный ход
4 end
5  $M \leftarrow$  getMoves( $s$ );
6 if уровень сложности = EASY then
7   |  $m^* \leftarrow$  случайный элемент  $M$ ;
8 else if уровень сложности = MEDIUM then
9   | сортировать  $M$  по убыванию evaluateAfterMove;
10  |  $m^* \leftarrow$  случайный элемент из первой половины  $M$ ;
11 else
12  | сортировать  $M$  по убыванию evaluateAfterMove;
13  |  $m^* \leftarrow$  случайный элемент из первых трёх элементов  $M$ ;
14 end
15 return  $m^*$ 
```

---

чение которого будет возвращено в качестве ответа. Благодаря этому, даже если лимит времени оказался исчерпан ранее, чем поиск на очередной глубине подошел к концу, всегда будет допустимый ответ, который можно будет использовать как ход алгоритма.

Максимальная глубина  $d_{\max}$  и лимит времени  $T$  зависят от выбранных пользователем уровня сложности алгоритма и размера игрового поля.

Пример выбора хода Минимаксом показан на рис. 8. Он построен на той же позиции, что и пример функции оценки на рис. 7.



**Рис. 8** — Фрагмент дерева Минимакса

Таблица 3.3

**Параметры минимакса (поле  $9 \times 9$ )**

| Уровень | $d_{\max}$ | $T$ , мс |
|---------|------------|----------|
| EASY    | 3          | 700      |
| MEDIUM  | 4          | 1800     |
| HARD    | 5          | 3600     |

После получения всего множества возможных вариантов хода из текущей позиции они сортируются с помощью функции оценки. Уже обработанные алгоритмом позиции и их оценки хранятся в таблице `cache`. Общая схема работы алгоритма приведена в алгоритме 2.

---

**Алгоритм 2: Минимакс с итеративным углублением**

---

```
1 Функция minimax( $s, d, isMax$ ):
2   if в cache есть значение для ( $s, d, isMax$ ) then
3     return значение из cache;
4   end
5   if terminal( $s$ ) then
6     return терминальную оценку позиции;
7   end
8   if  $d = 0$  then
9     return evaluate( $s$ );
10  end
11   $p \leftarrow$  игрок, которому принадлежит ход в узле  $s$ ;
12   $M \leftarrow$  getMoves( $s, p$ );
13  if  $isMax$  then
14     $best \leftarrow -\infty$ ;
15    foreach  $m \in M$  do
16       $best \leftarrow \max(best, \text{minimax}(\text{apply}(s, m), d - 1, \text{false}))$ ;
17    end
18  else
19     $best \leftarrow +\infty$ ;
20    foreach  $m \in M$  do
21       $best \leftarrow \min(best, \text{minimax}(\text{apply}(s, m), d - 1, \text{true}))$ ;
22    end
23  end
24  сохранить  $best$  в cache для ( $s, d, isMax$ );
25  return  $best$ ;

26  $m^* \leftarrow$  первый допустимый ход;
27 for  $d \leftarrow 1$  to  $d_{\max}$  do
28   if истёк лимит времени  $T$  then
29     break
30   end
31    $m^* \leftarrow \arg \max_{m \in M(s)} (\text{minimax}(\text{apply}(s, m), d - 1, \text{false}) +$ 
    movementPreference( $m, s$ ));
32 end
33 return  $m^*$ 
```

---

### 3.4. Альфа-бета отсечения

AlphaBetaAlgorithm является наиболее сложным из реализованных. Он представляет собой улучшенный Минимакс с несколькими независимыми механизмами ускорения: отсечениями ветвей дерева игры и сортировкой ходов [3].

#### Альфа-бета отсечение

Алгоритм использует два следующих параметра:  $\alpha$  — это наилучшая оценка, которая гарантирована максимизирующему игроку, а  $\beta$  — это наилучшая оценка, гарантированная минимизирующему. Если в ходе перебора оказывается, что  $\beta \leq \alpha$ , оставшиеся ходы в данном узле дерева можно не рассматривать: они не изменят итогового выбора. Это позволяет уменьшить число рассматриваемых состояний и благодаря этому достичь большей глубины поиска хода.

Схематический пример отсечения приведён на рис. 9. Используется тот же фрагмент дерева, что и на рис. 8.

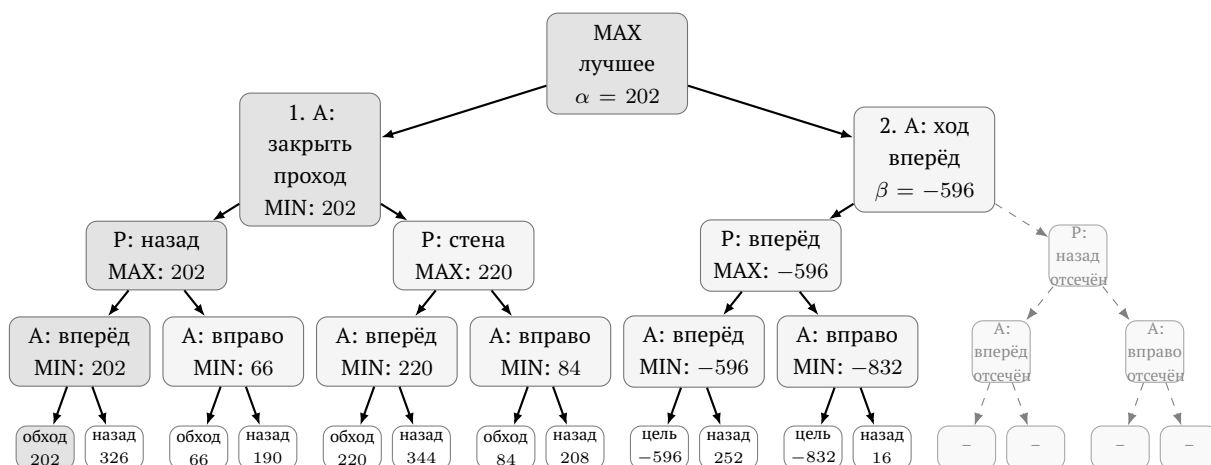


Рис. 9 — Фрагмент дерева альфа-бета отсечения

#### Сортировка ходов

Эффективность отсечений ветвей критически зависит от порядка рассмотрения ходов: если лучший ход рассматривается первым, отсечения происходят как можно раньше, что оптимизирует затраченное время. Функция сортировки scoreMove оценивает каждый ход до его применения по нескольким критериям:

- $+10\,000 / +8\,000$ , если ход является первым или вторым *killer-ходом* данного уровня глубины;
- вес из таблицы goodMoves — истории ходов, вызывавших отсечения в предыдущих узлах;
- значение функции оценки после применения хода.

Под *killer-ходом* понимается ход, который на проверке той же глубины поиска в дереве ранее уже привёл к отсечению  $\beta \leq \alpha$ . Такой ход считается перспективным не потому, что он обязательно является лучшим в текущей позиции, а потому что в похожих по глубине узлах он уже оказался достаточно сильным, чтобы сделать дальнейший перебор ветви бессмысленным. В реализации алгоритма для каждого уровня глубины хранится до двух таких ходов в историческом массиве `bestMoves`, при сортировке они получают высокий приоритет и рассматриваются раньше остальных.

Таблица `goodMoves` решает похожую задачу, но хранит не два последних хода для конкретной глубины, а накопленную статистику по строковому представлению ходов. Каждый раз, когда ход приводит к отсечению, его счётчик увеличивается. При следующих сортировках ход с наибольшим числом успешных отсечений получает небольшой дополнительный вес.

### Поиск стабильной терминальной позиции

Обычный поиск с ограничением глубины имеет одну важную проблему: он может остановиться в момент, когда позиция ещё не является стабильной. Например, алгоритм дошёл до предельной глубины и видит, что путь до цели стал короче, но не успел проверить, что соперник следующим ходом тоже может выйти на целевую линию или поставить решающую стену. В такой ситуации простая статическая оценка не может гарантировать победу алгоритма.

Для уменьшения этой ошибки в `AlphaBetaAlgorithm` используется поиск стабильной терминальной позиции. Его идея состоит в том, что на предельной глубине алгоритм не всегда сразу вызывает `evaluate()`, а сначала проверяет, не является ли позиция «острой». Таковой считается позиция, в которой хотя бы один из игроков находится не дальше двух ходов от победы.

Если позиция признана «острой», алгоритм выполняет небольшое дополнительное расширение дерева: рассматривает немедленные победные ходы и до восьми наиболее ценных установок стен, которые сильнее всего увеличивают путь соперника до цели. После этого для оставшихся «спокойных» позиций снова используется обычная функция оценки.

Таблица 3.4

**Параметры альфа-бета алгоритма для поля  $9 \times 9$**

| Уровень | $d_{\max}$ | $T$ , мс |
|---------|------------|----------|
| EASY    | 4          | 1300     |
| MEDIUM  | 6          | 3600     |
| HARD    | 8          | 7600     |

Общая схема работы альфа-бета алгоритма приведена в алгоритме 3.

---

**Алгоритм 3: Альфа-бета**

---

```
1 Функция alphabeta( $s, d, \alpha, \beta, isMax$ ):
2   if terminal( $s$ ) then
3     return терминальная оценка
4   end
5   if  $d = 0$  then
6     if isNoisyPosition( $s$ ) then
7       return quiescence( $s, \alpha, \beta, isMax, 2$ )
8     else
9       return  $F(s)$ 
10    end
11  end
12   $M \leftarrow \text{getMoves}(s)$ ;
13  best  $\leftarrow -\infty$  если isMax, иначе  $+\infty$ ;
14  foreach  $m \in M$  do
15     $v \leftarrow \text{alphabeta}(\text{apply}(s, m), d - 1, \alpha, \beta, \neg isMax)$ ;
16    обновить best,  $\alpha$  или  $\beta$ ;
17    if  $\beta \leq \alpha$  then
18      обновить killer-ходы и goodMoves;
19      break ;                                // отсечение
20    end
21  end
22  return best
```

---

### 3.5. Монте-Карло

Алгоритм MonteCarloAlgorithm строит статистику по многократным симуляциям игры с текущей позиции фишки на игровом поле до конца партии [9]. В отличие от уже рассмотренных алгоритмов, он не перебирает дерево равномерно до фиксированной глубины, а постепенно концентрирует свои вычисления на наиболее перспективных ветвях. Структура алгоритма — дерево узлов, каждый из которых соответствует некоторому состоянию игры и хранит счётчики посещений узла во время проверки  $n_v$  и побед  $w_v$ , которые были достигнуты с посещением этого узла.

Основной цикл алгоритма состоит из четырёх фаз.

#### 3.5.1. Выбор

Начиная с корня, алгоритм спускается по дереву, на каждом шаге выбирая дочерний узел с максимальным значением УСТ. Этот показатель нужен для баланса между двумя задачами: продолжать проверять ходы, которые уже часто



приводили к победе, а также исследовать менее изученные ходы.

$$\text{UCT}(v) = \frac{w_v}{n_v} + C \cdot \sqrt{\frac{\ln n_{\text{parent}}}{n_v}} + h(v).$$

Первое слагаемое  $\frac{w_v}{n_v}$  отражает уже накопленную статистику: чем чаще симуляции из узла  $v$  заканчивались победой, тем выше его оценка.

Второе слагаемое  $C \cdot \sqrt{\frac{\ln n_{\text{parent}}}{n_v}}$  отвечает за исследование. Если дочерний узел посещался редко, знаменатель  $n_v$  мал, поэтому значение этой части растёт и алгоритм получает стимул попробовать такой ход ещё раз. При увеличении числа посещений бонус постепенно уменьшается, и решение всё сильнее зависит от реальной статистики побед. Константа  $C = 1,2$  задаёт силу исследования: при большем значении алгоритм активнее проверяет разные ветви, при меньшем быстрее концентрируется на уже найденных хороших ходах.

Третье слагаемое  $h(v)$  является дополнительной эвристической поправкой, зависящей от функции оценки позиции:

$$h(v) = 0,15 \cdot \tanh(F(s_v)/500).$$

Она добавляет в формулу длины путей до цели, мобильность фишек игроков, оставшиеся лимиты стен и эндшпильные признаки. При этом коэффициент  $0,15$  и функция  $\tanh$  ограничивают влияние эвристики, чтобы она не подавляла статистику симуляций.

Если узел ни разу не посещался, ему присваивается приоритет  $+\infty$ . Благодаря этому каждый новый допустимый ход хотя бы один раз попадает в симуляцию.

### 3.5.2. Расширение

Когда все допустимые ходы из узла уже рассматривались в дочерних узлах, выбранный узел помечается как полностью расширенный. Иначе из списка ещё не рассмотренных ходов (предварительно отсортированных по `scoreMove`) берётся первый, создаётся новый дочерний узел и начинается его рассмотрение.

### 3.5.3. Симуляция

Из нового узла стартует случайная игра до терминального состояния или до исчерпания лимита шагов `rollOutDepth`. На каждом из ходов текущей симуляции применяется следующий механизм:

1. если есть победный ход, немедленно завершающий партию — используется он;
2. если соперник находится не далее 2 ходов от победы и у текущего игрока еще остались стены в запасе выбирается стена с максимальным `wallImpact`;

3. с вероятностью 0,22 выбирается случайный ход из полного списка;
4. иначе из случайной выборки размером 18 ходов выбирается один из четырёх лучших по scoreRolloutMove.

### 3.5.4. Обратное распространение

Результат сыгранной симуляции распространяется вверх: для каждого узла на пути от текущей позиции к корню дерева проставляются актуальные значения  $n_v$  и  $w_v$ .

### 3.5.5. Выбор лучшего хода

После истечения лимита времени или итераций алгоритма выбирается ход с наибольшим значением  $n_c + w_c/n_c$  среди дочерних узлов корня, что отдаёт предпочтение хорошо исследованным ходам.

Таблица 3.5

Параметры Монте-Карло для поля  $9 \times 9$

| Уровень | Бюджет итераций | $T$ , мс | Глубина симуляции |
|---------|-----------------|----------|-------------------|
| EASY    | 520             | 950      | $9 \cdot 4 = 36$  |
| MEDIUM  | 1700            | 2900     | $9 \cdot 7 = 63$  |
| HARD    | 3400            | 6200     | $9 \cdot 10 = 90$ |

Общая схема работы алгоритма Монте-Карло приведена в алгоритме 4.

#### Алгоритм 4: Монте-Карло

```

1  $r \leftarrow$  корневой узел с состоянием  $s$ ;
2 while не истёк лимит  $T$  и итераций  $< N$  do
3    $v \leftarrow \text{select}(r)$ ; // спуск по UCT
4   if  $\neg \text{terminal}(v.s)$  then
5      $v \leftarrow \text{expand}(v)$ ; // новый дочерний узел
6   end
7    $\text{result} \leftarrow \text{simulate}(v, \text{rolloutDepth})$ ;
8    $\text{backpropagate}(v, \text{result})$ ;
9   if  $r.n > 300$  и  $\max_c(w_c/n_c) > 0.97$  then
10    break
11  end
12 end
13 return  $\arg \max_{c \in \text{children}(r)} (n_c + w_c/n_c)$ 

```

Условие досрочного выхода из узла дерева ( $\text{bestWinRate} > 0.97$  при более чем 300 посещениях корня) позволяет экономить время в позициях, где один из ходов явно доминирует по статистике.

### 3.6. Сводные параметры сложности

Для удобства настройки экспериментов сравнения алгоритмов все уровни сложности сведены в таблицу 3.6. Значения приведены для стандартного поля  $9 \times 9$  как базовые параметры приложения. Для полей  $7 \times 7$  и  $11 \times 11$  параметры масштабируются в коде: на малом поле AlphaBeta может увеличивать глубину поиска, а на большом поле MiniMax и AlphaBeta уменьшают глубину либо увеличивают временной лимит, чтобы партия оставалась интерактивной.

Таблица 3.6

**Параметры уровней сложности алгоритмов для поля  $9 \times 9$**

| Алгоритм  | Уровень | Параметры                                                                 |
|-----------|---------|---------------------------------------------------------------------------|
| Random    | EASY    | Случайный выбор среди всех допустимых ходов                               |
| Random    | MEDIUM  | Случайный выбор из верхней половины ходов по функции оценки               |
| Random    | HARD    | Случайный выбор из трёх лучших ходов по функции оценки                    |
| MiniMax   | EASY    | Глубина 3, лимит 700 мс                                                   |
| MiniMax   | MEDIUM  | Глубина 4, лимит 1800 мс                                                  |
| MiniMax   | HARD    | Глубина 5, лимит 3600 мс                                                  |
| AlphaBeta | EASY    | Глубина 4, лимит 1300 мс, сортировка ходов                                |
| AlphaBeta | MEDIUM  | Глубина 6, лимит 3600 мс, приоритет ходов, ранее приводивших к отсечениям |
| AlphaBeta | HARD    | Глубина 8, лимит 7600 мс, поиск стабильности позиции в острых состояниях  |
| MCTS      | EASY    | До 520 итераций, лимит 950 мс, глубина rollout $9 \cdot 4 = 36$           |
| MCTS      | MEDIUM  | До 1700 итераций, лимит 2900 мс, глубина rollout $9 \cdot 7 = 63$         |
| MCTS      | HARD    | До 3400 итераций, лимит 6200 мс, глубина rollout $9 \cdot 10 = 90$        |

## 4. Сравнение алгоритмов

### 4.1. Методика сравнения

Сравнение алгоритмов выполняется в отдельном режиме игрового приложения. Цель добавления режима — получить не единичный результат отдельной партии, а агрегированную оценку поведения стратегий на одинаковых условиях.

В эксперименте участвуют все применяемые в игровом приложении «Коридор» алгоритмы. Они сравниваются попарно, количество различных пар равно 6.

Для каждой пары играется заданное пользователем число партий. Для компенсации преимущества первого хода роли игроков чередуются: часть партий алгоритм проводит за первого игрока, часть — за второго. Размер поля и уровень сложности каждого алгоритма задаются перед запуском эксперимента.

Важно отметить, что в режиме сравнения алгоритмов обучение весов функции оценки не выполняется.

### 4.2. Собираемые метрики

Для анализа недостаточно знать только победителя. Два алгоритма могут иметь близкую долю побед, но существенно отличаться по времени работы, глубине поиска или количеству просмотренных состояний игрового поля. Поэтому приложение собирает несколько параметров статистики.

Метрики глубины, узлов и отсечений особенно важны для сравнения алгоритмов `MiniMaxAlgorithm` и `AlphaBetaAlgorithm`. Оба алгоритма основаны на дереве игры, но альфа-бета должен просматривать меньше узлов при большей достижимой глубине за счёт отсечений. Для `MonteCarloAlgorithm` ключевыми являются число итераций и время хода, поскольку качество решения определяется количеством проведённых симуляций.

### 4.3. План эксперимента

Итоговый эксперимент проводится на трёх размерах поля:  $7 \times 7$ ,  $9 \times 9$  и  $11 \times 11$ . Для всех алгоритмов устанавливается уровень сложности HARD. Для каждой пары алгоритмов проводится 50 партий. Так как сравниваются четыре алгоритма, всего рассматривается шесть пар, а суммарное число партий для одного размера поля равно 300.

Каждый алгоритм участвует в трёх попарных сравнениях, поэтому итоговая статистика для одного алгоритма агрегируется по 150 партиям. Роли первого и

**Метрики сравнения алгоритмов**

| Метрика                 | Смысл                                                  |
|-------------------------|--------------------------------------------------------|
| Количество партий       | Общее число завершённых партий с участием алгоритма    |
| Победы и поражения      | Основная игровая результативность алгоритма            |
| Доля побед              | Доля побед алгоритма среди всех сыгранных им партий    |
| Среднее число ходов     | Косвенная характеристика стиля игры и сложности партий |
| Среднее время хода      | Практическая вычислительная стоимость алгоритма        |
| Средняя глубина         | Насколько глубоко алгоритм успевает просмотреть дерево |
| Среднее число узлов     | Объём реально просмотренного дерева поиска             |
| Среднее число отсечений | Эффективность alpha-beta оптимизации                   |

второго игрока чередуются между партиями, поэтому при 50 партиях на пару каждый алгоритм начинает одинаковое число раз.

#### **4.4. Условия проведения эксперимента**

Описанный эксперимент проводился на ноутбуке MacBook Pro с процессором Apple M3 Pro, включающим 11 ядер: 5 производительных и 6 энергоэффективных. Объём оперативной памяти составлял 18 ГБ. Операционная система — macOS 15.0.

Игровое приложение запускалось на Java 17, использовалась сборка Amazon Corretto 17.0.10 для архитектуры arm64. Серии партий выполнялись в одном экземпляре приложения последовательно.

#### **4.5. Таблицы результатов**

##### **4.5.1. Результаты эксперимента на большом поле**

Итоговая статистика представляется в двух формах. Первая форма — стандартная турнирная таблица. В ячейках таблицы 4.2 указаны победы, поражения и партии, завершившиеся по лимиту ходов, для алгоритма в соответствующей строке.

Вторая форма — агрегированная таблица по каждому алгоритму. Для читаемости она разделена на две части: показатели качества игры и вычислительные показатели.

Таблица 4.2

**Турнирная таблица попарных результатов**

| Алгоритм   | Случайный | MiniMax | MonteCarlo | AlphaBeta |
|------------|-----------|---------|------------|-----------|
| Случайный  | —         | 0–48–2  | 0–48–2     | 0–49–1    |
| MiniMax    | 48–0–2    | —       | 26–23–1    | 10–39–1   |
| MonteCarlo | 48–0–2    | 23–26–1 | —          | 17–33–0   |
| AlphaBeta  | 49–0–1    | 39–10–1 | 33–17–0    | —         |

В таблице качества игры столбец «Партий» показывает число партий, в которых участвовал алгоритм. «Победы» и «Поражения» отражают результат партии независимо от того, первым или вторым игроком выступал алгоритм. «Лимит» означает партии, остановленные по максимальному числу ходов без достижения целевой линии. «Доля побед» — отношение числа побед к общему числу партий.

Таблица 4.3

**Сводная статистика качества игры**

| Алгоритм   | Партий | Победы | Поражения | Лимит | Доля побед |
|------------|--------|--------|-----------|-------|------------|
| Случайный  | 150    | 0      | 145       | 5     | 0,00%      |
| MiniMax    | 150    | 84     | 62        | 4     | 56,00%     |
| MonteCarlo | 150    | 88     | 59        | 3     | 58,67%     |
| AlphaBeta  | 150    | 121    | 27        | 2     | 80,67%     |

Вычислительные показатели позволяют оценить цену найденного качества игры. Длина партии показывает среднее число ходов до завершения игры, время хода — среднее время выбора одного действия, глубина поиска — глубину рассмотрения игрового дерева, а число состояний — средний объём просмотренных позиций на поле. Последняя строка показывает, сколько ветвей в среднем было отброшено эвристиками или альфа-бета отсечением.

Таблица 4.4

**Сводная статистика вычислительных показателей**

| Показатель            | Случайный | MiniMax | MonteCarlo | AlphaBeta |
|-----------------------|-----------|---------|------------|-----------|
| Длина партии, ходов   | 114,87    | 111,43  | 120,27     | 107,17    |
| Время хода, мс        | 1,0       | 1578,5  | 6576,2     | 2743,9    |
| Глубина поиска        | 1,00      | 3,96    | 108,60     | 6,71      |
| Состояний просмотрено | 4,4       | 2158,1  | 444,7      | 5405,1    |
| Ветвей отброшено      | 0,0       | 0,0     | 0,0        | 1140,3    |

**4.5.2. Результаты эксперимента на малых полях**

Также приведены таблицы результатов эксперимента на меньших по размеру полях.

Таблица 4.5

**Сводная статистика качества игры на поле  $7 \times 7$** 

| Алгоритм   | Партий | Победы | Поражения | Лимит | Доля побед |
|------------|--------|--------|-----------|-------|------------|
| Случайный  | 150    | 0      | 150       | 0     | 0,00%      |
| MiniMax    | 150    | 126    | 24        | 0     | 84,00%     |
| MonteCarlo | 150    | 81     | 69        | 0     | 54,00%     |
| AlphaBeta  | 150    | 93     | 57        | 0     | 62,00%     |

Таблица 4.6

**Сводная статистика вычислительных показателей на поле  $7 \times 7$** 

| Показатель            | Случайный | MiniMax | MonteCarlo | AlphaBeta |
|-----------------------|-----------|---------|------------|-----------|
| Длина партии, ходов   | 58,03     | 46,10   | 58,70      | 45,70     |
| Время хода, мс        | 0,4       | 737,6   | 4807,6     | 2274,3    |
| Глубина поиска        | 0,99      | 4,81    | 68,73      | 7,50      |
| Состояний просмотрено | 4,4       | 8288,4  | 797,7      | 13210,3   |
| Ветвей отброшено      | 0,0       | 0,0     | 0,0        | 2785,9    |

Таблица 4.7

**Сводная статистика качества игры на поле  $9 \times 9$** 

| Алгоритм   | Партий | Победы | Поражения | Лимит | Доля побед |
|------------|--------|--------|-----------|-------|------------|
| Случайный  | 150    | 0      | 148       | 2     | 0,00%      |
| MiniMax    | 150    | 97     | 52        | 1     | 64,67%     |
| MonteCarlo | 150    | 107    | 43        | 0     | 71,33%     |
| AlphaBeta  | 150    | 94     | 55        | 1     | 62,67%     |

Таблица 4.8

**Сводная статистика вычислительных показателей на поле  $9 \times 9$** 

| Показатель            | Случайный | MiniMax | MonteCarlo | AlphaBeta |
|-----------------------|-----------|---------|------------|-----------|
| Длина партии, ходов   | 97,27     | 73,23   | 97,37      | 96,67     |
| Время хода, мс        | 0,7       | 935,2   | 4991,9     | 2277,5    |
| Глубина поиска        | 1,00      | 4,87    | 88,71      | 7,59      |
| Состояний просмотрено | 4,5       | 5993,4  | 588,4      | 7327,6    |
| Ветвей отброшено      | 0,0       | 0,0     | 0,0        | 1510,9    |

#### 4.5.3. Зависимость вычислительной стоимости от размера поля

На рис. 10, 11 и 12 показаны зависимости средней длины партии, времени выбора хода и количества посещенных узлов дерева от размера выбранного игрового поля. Для каждого графика используются агрегированные средние значения с участием соответствующего алгоритма.

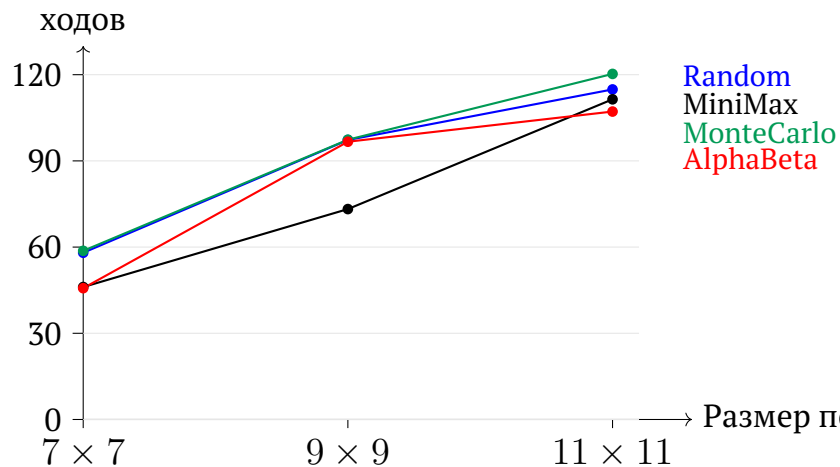


Рис. 10 — Средняя длина партии при изменении размера поля

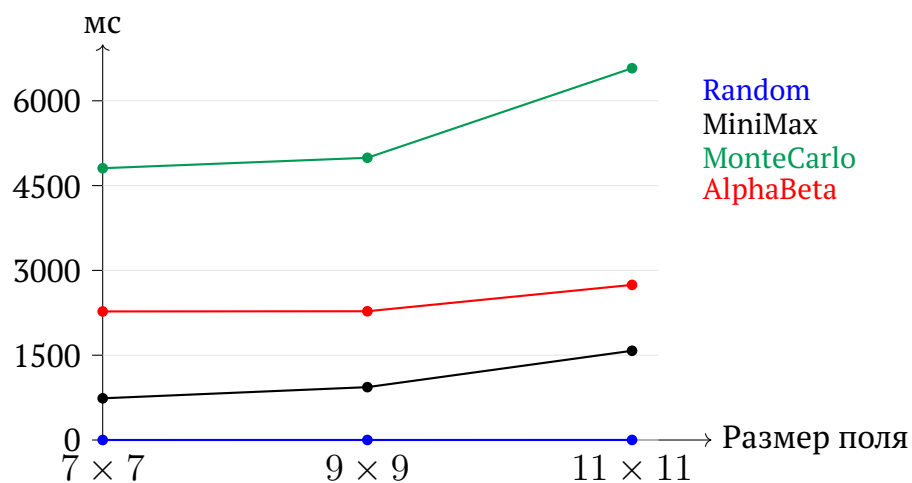
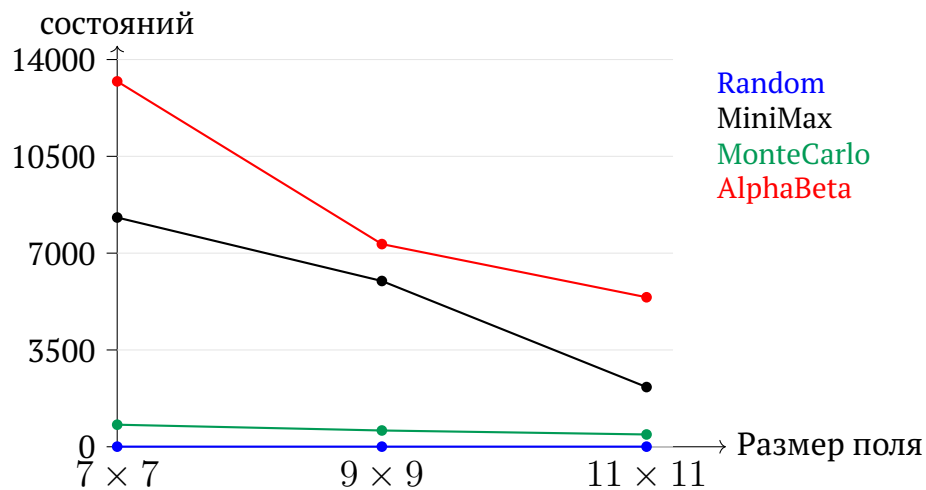


Рис. 11 — Среднее время выбора хода при изменении размера поля

MonteCarlo тратит больше всего времени на ход из-за большого числа симуляций.





**Рис. 12** — Среднее число просмотренных игровых позиций при изменении размера поля

Для случайного алгоритма время хода и число просмотренных состояний остаются минимальными, поскольку он не строит полноценное дерево игры. Алгоритмы MiniMax и AlphaBeta просматривают значительно больше состояний на малых и средних полях, потому что за отведённое время успевают глубже раскрыть дерево вариантов, ведь с увеличением размеров игрового поля растёт «стоимость» рассмотрения игровой позиции.

## 4.6. Сравнение весов функции оценки

Отдельный эксперимент с весами функции оценки используется не как окончательное доказательство превосходства обученной версии, а как проверка того, что механизм коррекции не ухудшает поведение алгоритма в одинаковых условиях. Один и тот же алгоритм запускается против своей копии, причем одна копия использует начальные вручную заданные веса `EvaluationWeights.defaults()`, а другая — веса, полученные после self-play обучения и сохранённые в `evaluationweights.properties`.

Играется 50 партий на поле  $11 \times 11$  с одинаковым уровнем сложности HARD для обеих вариаций алгоритма, роли первого и второго игрока чередуются. Результат сравнения и вычислительный контекст эксперимента показаны на рис. 13.

| Результат партий |           | Вычислительный контекст |             |
|------------------|-----------|-------------------------|-------------|
| Начальные веса   | 19 побед  | Длина партии            | 117,84 хода |
| Обновлённые веса | 27 побед  | Время хода              | 2903,7 мс   |
| Лимит ходов      | 4 партии  | Глубина поиска          | 6,66        |
| Всего            | 50 партий | Состояния               | 5213,9      |
|                  |           | Отсечения               | 1046,5      |

**Рис. 13** — Сравнение начальных и обновлённых весов функции оценки

## 4.7. Анализ результатов

По итогам основного эксперимента на поле  $11 \times 11$  наибольшую долю побед показал AlphaBetaAlgorithm: 121 победа из 150 партий, что соответствует 80,67% побед. Алгоритм уверенно выиграл все три попарных сравнения. При этом среднее время хода AlphaBeta меньше, чем у MonteCarlo, но на ход тратится больше, чем у MiniMax и случайного алгоритма.

MiniMax и MonteCarlo показали близкие результаты по доле побед: 56,00% и 58,67% соответственно. В прямом сравнении MiniMax выиграл 26 партий, в то время как MonteCarlo — 23, одна партия завершилась по лимиту ходов. Это показывает, что статистически их игровая сила в выбранных условиях близка, однако MonteCarlo требует заметно большего времени на ход.

Случайный алгоритм не выиграл ни одной партии. Тем не менее несколько партий с его участием завершились по лимиту ходов, что связано с большими размерами поля  $11 \times 11$  и числом возможных ходов.

Таким образом, в рамках обозначенного сравнения наиболее эффективным оказался алгоритм AlphaBeta: он сочетает максимальный процент побед с приемлемой вычислительной стоимостью.

## 5. Игровое приложение

### 5.1. Техническая реализация

Игровое приложение «Коридор» написано на языке программирования Java версии 17 с использованием фреймворка Spring Boot версии 3.2.5, а также библиотеки Vaadin версии 24.4.2 [10], которая отвечает за веб-интерфейс. Для сокращения шаблонного кода используется библиотека Lombok. Система сборки кода — Maven.

Архитектура приложения разделена на несколько независимых слоёв. Общая схема взаимодействия слоёв представлена на рис. 14.

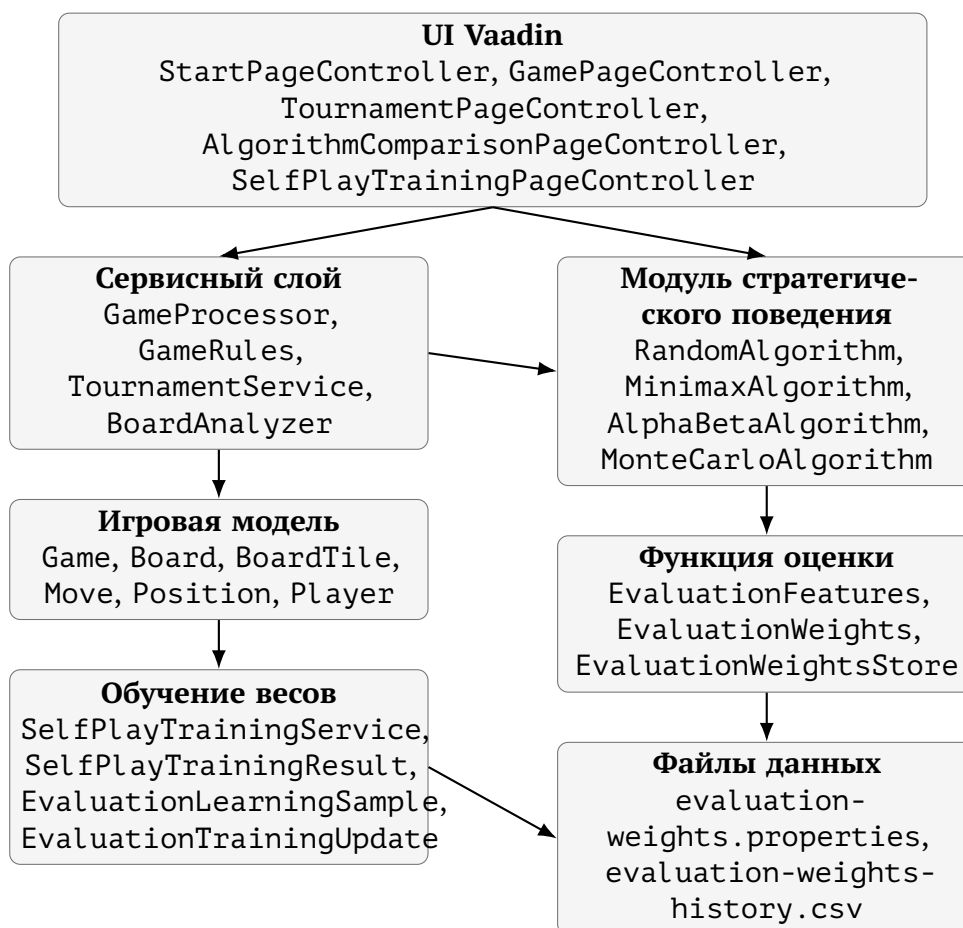


Рис. 14 — Общая архитектура приложения

*Модель* содержит классы, описывающие игровые объекты. Класс `Position` хранит координаты клетки  $(i, j)$  и предоставляет метод `move()` для вычисления соседней позиции. Класс `BoardTile` описывает одну клетку поля четырьмя булевыми флагами доступности переходов. Класс `Board` хранит граф поля в виде `HashMap<Position, BoardTile>`, позиции фишек обоих игроков и реализует

все операции с доской: размещение перегородок (`placeWall`), получение доступных ходов (`getAvailableMoves`) и полное копирование (`copy`). Класс `Move` описывает ход: его тип (`MOVE_PLAYER` или `PLACE_WALL`), идентификатор игрока и позиции начала и конца перемещения или стены. Класс `Game` хранит конфигурацию текущей партии: размер игрового поля (`GameSize`), объекты игроков, номер текущего хода, время начала игры и флаг ее завершения.

*Игроки* реализуют иерархию через абстрактный класс `Player` с методом `getMove`. Класс `HumanPlayer` возвращает `null`, поскольку ход игрока получается через его взаимодействие с интерфейсом игрового приложения. Специальный класс `ComputerPlayer` делегирует вызов конкретному объекту `Algorithm`. Фабрика `ComputerPlayerFabric` создаёт нужную реализацию алгоритма по типу и уровню сложности в соответствии с предпочтениями пользователя приложения.

*Алгоритмы* описаны в главе 2. Каждый из них реализует единый интерфейс `Algorithm`. Класс `AlgorithmReport` является неизменяемой записью с результатами последнего хода: выбранный ход, его оценка, достигнутая глубина поиска, количество рассмотренных узлов, количество отсечений ветвей дерева игры, время в миллисекундах, затраченное на анализ хода и его текстовое пояснение. Класс `EvaluationFeatures` хранит вектор признаков состояния; `EvaluationWeights` — веса; `EvaluationWeightsStore` управляет загрузкой и сохранением весов для историчности.

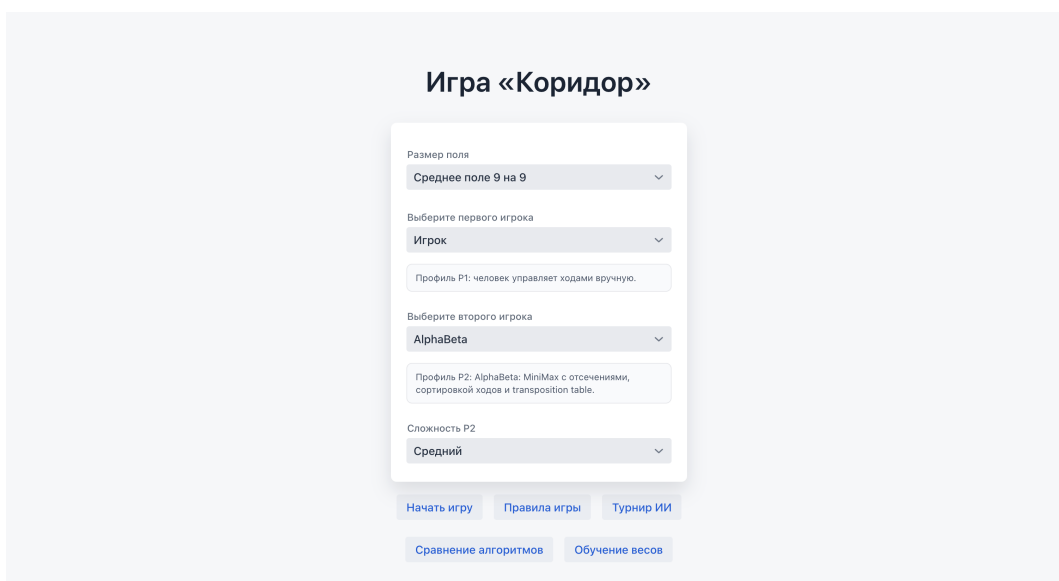
*Сервисный слой* включает несколько компонентов. `BoardFactory` создаёт начальное состояние доски заданного размера. Класс `GameRules` проверяет корректность установки стены и конвертирует координаты поля на пользовательском интерфейса в игровые координаты, понятные алгоритмам. `GameProcessor` — аннотированный `@SessionScope` Spring-компонент, управляющий всем жизненным циклом партии: запуск (`startNewGame`), выполнение ходов (`makeMove`, `makeComputerMove`), сбор образцов для обучения весов и получение подсказок от всех четырёх алгоритмов в случае запроса таковых от пользователя приложения. `BoardAnalyzer` формирует текстовые пояснения к ходам ИИ. Класс `TournamentService` позволяет запускать автоматические серии партий для турнира и сравнения алгоритмов. `SelfPlayTrainingService` отвечает за обучение весов параметров функции оценки: проводит партии между алгоритмами, собирает объекты `EvaluationLearningSample`, передаёт их в отдельный класс `EvaluationWeightsStore` и возвращает результат обучения в виде объекта класса `SelfPlayTrainingResult`.

*Интерфейсный слой* реализован через библиотечные компоненты `Vaadin`. `StartPageController` отображает стартовый экран с выбором режима и параметров. `GamePageController` реализует основной экран игры: отрисовку игровой доски в виде сетки, историю ходов (`GameHistoryPanel`), кнопки управления. `TournamentPageController`, `AlgorithmComparisonPageController` и

SelfPlayTrainingPageController реализуют отображение режимов турнира, сравнения алгоритмов и обучения весов функции оценки.

## 5.2. Взаимодействие с приложением

При запуске игрового приложения «Коридор» пользователь попадает на стартовый экран, на котором настраиваются параметры предстоящей партии (рис. 15).



**Рис. 15** — Стартовый экран выбора режима игры и алгоритмов

Доступны три варианта размера игрового поля: малое поле  $7 \times 7$  с 8 стенами у каждого игрока, стандартное  $9 \times 9$  с 10 стенами, большое  $11 \times 11$  с 12 стенами. Размер поля влияет на все параметры алгоритмов: лимиты времени и максимальные глубины масштабируются пропорционально.

Первым игроком может управлять человек или один из четырёх алгоритмов программного соперника. Если выбран алгоритм, дополнительно настраивается его уровень сложности.

Второй игрок всегда управляется алгоритмом с настройкой его сложности. Карточки профилей отображают краткое описание выбранной стратегии.

На игровом экране пользователь видит несколько областей. Пример этого экрана приведён на рис. 16.

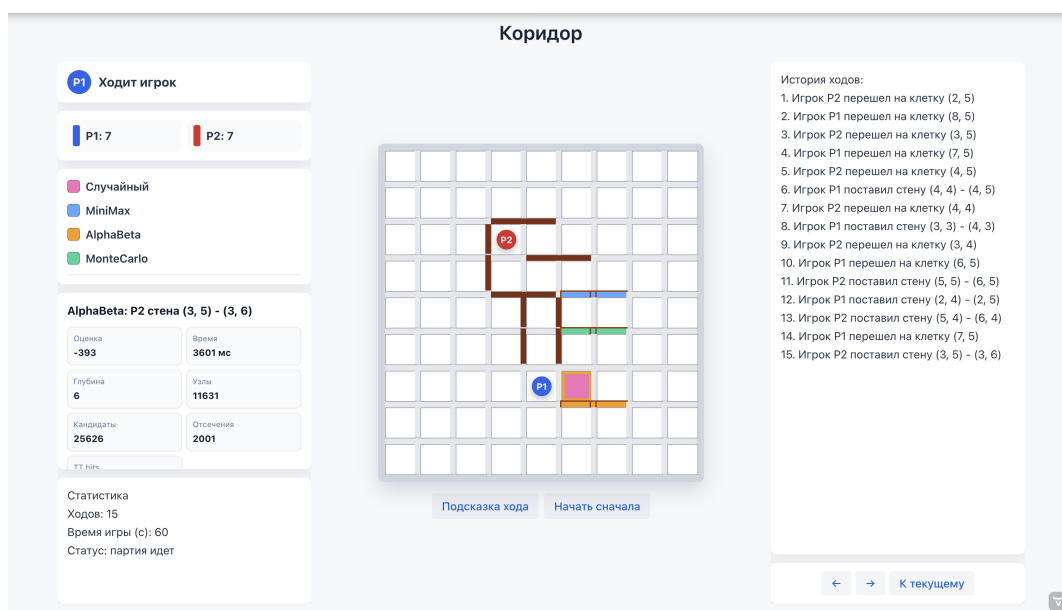
Игровое поле отрисовывается в виде сетки: клетки чередуются с промежутками, в которых могут быть размещены стены, блокирующие передвижения между ними. Каждая клетка имеет размер 42–50 пикселей в зависимости от размера поля. Фишки отображаются цветными кружками разного цвета. Допустимые для перемещения клетки подсвечиваются зелёным при наведении. При наведении на щель между клетками предварительно подсвечивается потенциальная стена.

Перемещение фишки игрока выполняется кликом на целевую клетку. Если ход недопустим, появляется уведомление.

Установка стены выполняется кликом на промежуток между клетками. GameRules.extractWall определяет по координатам интерфейса начальную и конечную клетки перегородки. Затем GameRules.canPlaceWall копирует доску, устанавливает стену и проверяет достижимость целевых линий обоих игроков через обход в ширину.

В режиме «Игрок против ИИ» после хода пользователя автоматически запускается выбранный алгоритм. В режиме «ИИ против ИИ» каждый ход требует нажатия кнопки «Сделать ход ИИ», что позволяет наблюдать за партией пошагово, а не видеть лишь результат игры. Рядом с кнопкой отображается, чья очередь делать ход.

Кнопка «Подсказка хода» запускает все четыре алгоритма на текущей позиции и подсвечивает рекомендованные ими ходы разными цветами, каждый из которых соответствует одному из алгоритмов. Пример отображения подсказок приведён на рис. 16.



**Рис. 16** — Отображение подсказок от алгоритмов на игровом поле

Панель информации отображает: чья очередь хода, количество оставшихся стен у каждого игрока, соответствие цветов и алгоритмов, а также отчёт о последнем ходе ИИ. Помимо прочего можно наблюдать за статистикой партии: числом ходов и затраченном времени.

С помощью кнопок «←», «→» и «К текущему» можно смотреть историю ходов. Она также дублируется текстовым списком в правой панели.

Турнирный режим позволяет выбрать два алгоритма, их уровни сложности, размер поля и число партий. Результаты выводятся в режиме реального времени: по завершении каждой партии обновляется счётчик побед. По завершении серии

выводится сводная таблица. Пример работы режима показан на рис. 17.

Турнир ИИ

ИИ P1: MiniMax, ИИ P2: AlphaBeta, Сложность P1: Сложный, Сложность P2: Легкий

Размер поля: Большое поле 11 на 11, Количество партий: 6

Запустить турнир, Назад

Старт турнира: MiniMax vs AlphaBeta  
Поле: Большое поле 11 на 11, P1 сложность: Сложный, P2 сложность: Легкий, партий: 6  
Партия 1/6: начинает P1, после нее останется партий: 5  
Партия 1/6, ход 1/300, до лимита партии: 299, после партии останется партий: 5, последний ход: P1  
Партия 1/6, ход 2/300, до лимита партии: 298, после партии останется партий: 5, последний ход: P2  
Партия 1/6, ход 3/300, до лимита партии: 297, после партии останется партий: 5, последний ход: P1  
Партия 1/6, ход 4/300, до лимита партии: 296, после партии останется партий: 5, последний ход: P2  
Партия 1/6, ход 5/300, до лимита партии: 295, после партии останется партий: 5, последний ход: P1

Рис. 17 — Экран турнирного режима ИИ

Режим сравнения алгоритмов автоматически проводит серию матчей между всеми попарными комбинациями четырёх алгоритмов и выводит сводную таблицу итогов с процентом побед, средним числом ходов, средним временем хода, средней глубиной поиска, числом узлов и отсечений. Интерфейс режима приведён на рис. 18.

Сравнение алгоритмов

Сложность Random: Средний, Сложность MiniMax: Средний, Сложность MonteCarlo: Средний, Сложность AlphaBeta: Средний, Размер поля: Среднее поле 9 на 9

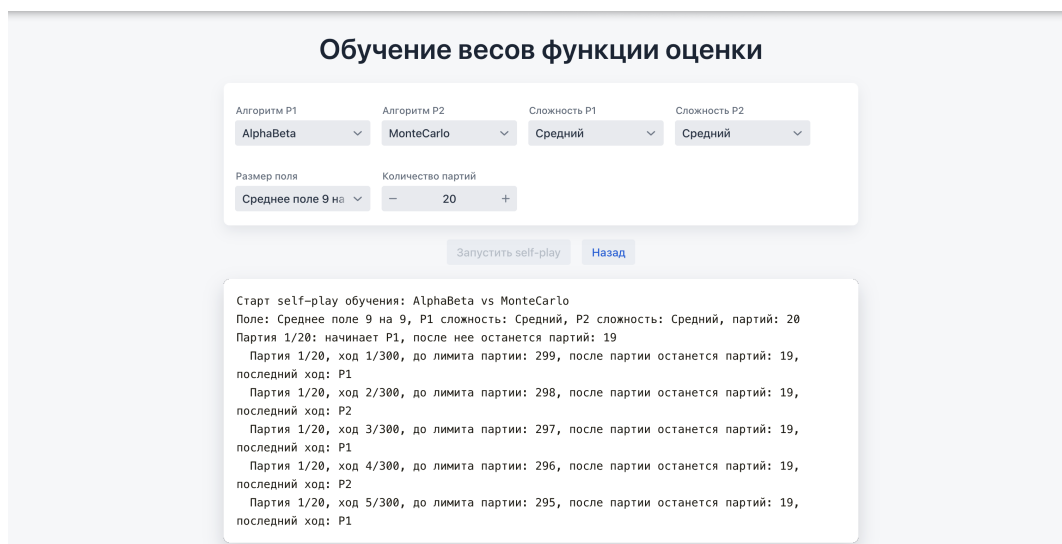
Партий на пару: 10

Сравнить, Назад

Сравнение 1/6: Случайный vs MiniMax  
Сравнение 2/6: Случайный vs MonteCarlo  
Сравнение 3/6: Случайный vs AlphaBeta  
Сравнение 4/6: MiniMax vs MonteCarlo  
Сравнение 5/6: MiniMax vs AlphaBeta

Рис. 18 — Экран сравнения алгоритмов

Режим обучения весов запускает серию партий между двумя выбранными алгоритмами. По завершении каждой партии сервис обучения обновляет веса функции оценки, сохраняет их в файл и выводит ход эксперимента. Пример обучения показан на рис. 19.



**Рис. 19** — Экран self-play обучения весов функции оценки



## Заключение

В ходе выполнения выпускной квалификационной работы была исследована настольная логическая игра «Коридор». Реализованы алгоритмы проверки корректности перемещения фишки с учётом всех правил прыжков через соперника и проверки корректности установки стены с помощью обхода графа в ширину. Реализован алгоритм поиска кратчайшего пути до целевой линии с восстановлением пути.

Разработана функция оценки позиции, учитывающая семь параметров. Реализован механизм коррекции весов функции оценки по результатам завершённых партий с сохранением в файл конфигурации между сессиями.

Реализованы четыре алгоритма программного соперника с тремя уровнями сложности каждый.

Разработано игровое приложение с режимами «Игрок против ИИ», «ИИ против ИИ», режимом подсказок одновременно от всех четырёх алгоритмов, просмотром истории партии, турнирным режимом и режимом сравнения алгоритмов.

## Список литературы

- [1] BoardGameGeek. Games 100 [Электронный ресурс]. — Режим доступа: [https://boardgamegeek.com/wiki/page/Games\\_100](https://boardgamegeek.com/wiki/page/Games_100) (дата обращения: 02.04.2026).
- [2] Games, Hobby. Коридор. Правила игры [Электронный ресурс]. — Режим доступа: [https://hobbygames.ru/download/rules/Quoridor\\_Rules.pdf](https://hobbygames.ru/download/rules/Quoridor_Rules.pdf) (дата обращения: 03.04.2026).
- [3] Russell, S. Artificial Intelligence: A Modern Approach [Текст] / S. Russell, P. Norvig. — Hoboken : Pearson, 2021.
- [4] Introduction to Algorithms [Текст] / T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. — 4 изд. — Cambridge : The MIT Press, 2022.
- [5] Pearl, J. Heuristics: Intelligent Search Strategies for Computer Problem Solving [Текст] / J. Pearl. — Reading : Addison-Wesley, 1984.
- [6] Berlekamp, E. R. Winning Ways for Your Mathematical Plays [Текст] / E. R. Berlekamp, J. H. Conway, R. K. Guy. — 2 изд. — Natick : A K Peters, 2001.
- [7] Massague Respall, V. Monte Carlo Tree Search for Quoridor [Текст] / V. Massague Respall, J. A. Brown, H. Aslam // Proceedings of the 19th International Conference on Intelligent Games and Simulation, GAME-ON 2018. — 2018.
- [8] Quoridor-AI [Текст] / C. Jose, S. Kulshrestha, C. Ling и др. // Research report. — 2022.
- [9] Coulom, R. Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search [Текст] / R. Coulom // Computers and Games. — 2006. — С. 72–83.
- [10] Vaadin. Vaadin Flow Documentation [Электронный ресурс]. — Режим доступа: <https://vaadin.com/docs> (дата обращения: 05.03.2026).

## Реализация алгоритма MiniMax

Листинг А.1 — Класс MinimaxAlgorithm

```

1 public class MinimaxAlgorithm implements Algorithm {
2     private final Map<String, Integer> cache = new HashMap<>();
3     private AlgorithmReport lastReport;
4     private long nodesVisited;
5     private long consideredMoves;
6     private List<Position> recentPositions = List.of();
7
8     @Override
9     public ComputerAlgorithmType getType() {
10         return ComputerAlgorithmType.MINIMAX;
11     }
12
13     //отдает отчет о последнем ходе
14     @Override
15     public AlgorithmReport getLastReport() {
16         return lastReport;
17     }
18
19     //запоминает позиции игрока
20     @Override
21     public void setRecentPositions(List<Position>
22         recentPositions) {
23         this.recentPositions = recentPositions == null ? List.
24             of() : List.copyOf(recentPositions);
25     }
26
27     //ищет лучший ход итеративным углублением минимакса
28     @Override
29     public Move getMove(Board board,
30         ComputerPlayerHardnessLevel hardnessLevel, int playerId,
31         int wallsLeft1, int wallsLeft2) {
32         long startedAt = System.currentTimeMillis();
33         int size = (int) Math.sqrt(board.getTiles().size());
34         long timeLimit = getTimeLimit(hardnessLevel, size);

```

```

31     long endTime = System.currentTimeMillis() + timeLimit;
32     int maxDepth = getMaxDepth(hardnessLevel, size);
33
34     cache.clear();
35     nodesVisited = 0;
36     consideredMoves = 0;
37
38     int currentWalls = playerId == 1 ? wallsLeft1 :
        wallsLeft2;
39     Move tacticalMove = findEndgameMove(board, playerId,
        currentWalls);
40     if (tacticalMove != null) {
41         int tacticalScore = scoreMoveAfterApply(board,
            tacticalMove, playerId, size, wallsLeft1,
            wallsLeft2);
42         lastReport = new AlgorithmReport(getType().
            getDescription(), tacticalMove, tacticalScore,
            0, nodesVisited, 1, 0, 0, System.
            currentTimeMillis() - startedAt, "MiniMax_примен
            ил_эндшпильное_правило:_немедленная_победа_или_с
            рочная_блокировка");
43         return tacticalMove;
44     }
45
46     Move best = null;
47     int bestScore = 0;
48     int reachedDepth = 0;
49     for (int depth = 1; depth <= maxDepth; depth++) {
50         if (System.currentTimeMillis() > endTime) {
51             break;
52         }
53         SearchResult result = search(board, depth, playerId
            , size, wallsLeft1, wallsLeft2, endTime);
54         if (result.move() != null) {
55             best = result.move();
56             bestScore = result.score();
57             reachedDepth = depth;
58         }
59     }
60     lastReport = new AlgorithmReport(getType().

```

```

        getDescription(), best, bestScore, reachedDepth,
        nodesVisited, consideredMoves, 0, 0, System.
        currentTimeMillis() - startedAt, "MiniMax_перебрал_д
        ерево_без_alpha-beta_отсечений;_лимит_времени:_ " +
        timeLimit + "_мс");
61         return best;
62     }
63
64     // подбирает максимальную глубину поиска под выбранные сложность и
размер поля
65     private int getMaxDepth(ComputerPlayerHardnessLevel level,
        int size) {
66         boolean smallBoard = size <= GameSize.SMALL.
            getAmountOfTilesPerSide();
67         boolean largeBoard = size > GameSize.NORMAL.
            getAmountOfTilesPerSide();
68         return switch (level) {
69             case EASY -> 3;
70             case MEDIUM -> smallBoard ? 5 : 4;
71             case HARD -> largeBoard ? 4 : 5;
72         };
73     }
74
75     // ограничивает время на ход
76     @Override
77     public long getTimeLimit(ComputerPlayerHardnessLevel level,
        int size) {
78         boolean largeBoard = size > GameSize.NORMAL.
            getAmountOfTilesPerSide();
79         return switch (level) {
80             case EASY -> largeBoard ? 900L : 700L;
81             case MEDIUM -> largeBoard ? 2200L : 1800L;
82             case HARD -> largeBoard ? 4200L : 3600L;
83         };
84     }
85
86     // перебирает возможные первые ходы и выбирает ход с лучшей оценкой
87     private SearchResult search(Board board, int depth, int
        playerId, int size, int wallsLeft1, int wallsLeft2, long
        endTime) {

```

```

88      int currentWalls = playerId == 1 ? wallsLeft1 :
        wallsLeft2;
89      List<Move> moves = orderedLimitedMoves(board, getMoves(
        board, playerId, currentWalls), playerId, size,
        wallsLeft1, wallsLeft2);
90      consideredMoves += moves.size();
91
92      Move bestMove = null;
93      int bestValue = Integer.MIN_VALUE;
94      for (Move move : moves) {
95          if (System.currentTimeMillis() > endTime) {
96              return new SearchResult(bestMove, bestValue);
97          }
98
99          Board copy = board.copy();
100         applyMove(copy, move);
101
102         int newWallsLeft1 = wallsLeft1, newWallsLeft2 =
            wallsLeft2;
103         if (move.getMoveType() == MoveType.PLACE_WALL) {
104             if (playerId == 1) {
105                 --newWallsLeft1;
106             } else {
107                 --newWallsLeft2;
108             }
109         }
110
111         int value = minimax(copy, depth - 1, false,
112             playerId, size, newWallsLeft1,
113             newWallsLeft2, endTime);
114         value += movementPreference(move, board, playerId,
115             size, recentPositions);
116         if (value > bestValue) {
117             bestValue = value;
118             bestMove = move;
119         }
120     }
121     return new SearchResult(bestMove, bestValue);

```

```

122      //рекурсивно строит дерево игры - свой ход максимизирует оценку, ход
      соперника минимизирует
123      private int minimax(Board board, int depth, boolean
      maximizing, int playerId, int size, int wallsLeft1, int
      wallsLeft2, long endTime) {
124          nodesVisited++;
125          if (System.currentTimeMillis() > endTime) {
126              return evaluate(board, playerId, size, wallsLeft1,
              wallsLeft2);
127          }
128          String key = boardKey(board) + "/" + depth + "/" +
              maximizing + "/" + wallsLeft1 + "/" + wallsLeft2;
129          if (cache.containsKey(key)) {
130              return cache.get(key);
131          }
132
133          Integer terminal = terminalScore(board, playerId, size,
              depth);
134          if (terminal != null) {
135              cache.put(key, terminal);
136              return terminal;
137          }
138
139          if (depth == 0) {
140              int val = evaluate(board, playerId, size,
              wallsLeft1, wallsLeft2);
141              cache.put(key, val);
142              return val;
143          }
144
145          int current = maximizing ? playerId : (3 - playerId);
146          int walls = current == 1 ? wallsLeft1 : wallsLeft2;
147          List<Move> moves = orderedLimitedMoves(board, getMoves(
              board, current, walls), playerId, size, wallsLeft1,
              wallsLeft2);
148          consideredMoves += moves.size();
149
150          int best = maximizing ? Integer.MIN_VALUE : Integer.
              MAX_VALUE;
151          for (Move move : moves) {

```

```

152         Board copy = board.copy();
153         applyMove(copy, move);
154
155         int newWallsLeft1 = wallsLeft1, newWallsLeft2 =
            wallsLeft2;
156         if (move.getMoveType() == MoveType.PLACE_WALL) {
157             if (current == 1) {
158                 --newWallsLeft1;
159             } else {
160                 --newWallsLeft2;
161             }
162         }
163
164         int val = minimax(copy, depth - 1, !maximizing,
            playerId, size, newWallsLeft1, newWallsLeft2,
            endTime);
165         if (maximizing) {
166             best = Math.max(best, val);
167         } else {
168             best = Math.min(best, val);
169         }
170     }
171     cache.put(key, best);
172     return best;
173 }
174
175     // сортирует ходы по быстрой оценке и оставляет самые перспективные
176     private List<Move> orderedLimitedMoves(Board board, List<
        Move> moves, int playerId, int size, int wallsLeft1, int
        wallsLeft2) {
177         List<Move> ordered = new ArrayList<>(moves);
178         ordered.sort((a, b) -> Integer.compare(
179             scoreMoveForOrdering(board, b, playerId, size,
                wallsLeft1, wallsLeft2),
180             scoreMoveForOrdering(board, a, playerId, size,
                wallsLeft1, wallsLeft2)
181         ));
182         return ordered;
183     }
184

```



```

185     // дает ходу предварительный балл для сортировки перед глубоким поиском
186     private int scoreMoveForOrdering(Board board, Move move,
        int playerId, int size, int wallsLeft1, int wallsLeft2)
        {
187         return scoreMoveAfterApply(board, move, playerId, size,
            wallsLeft1, wallsLeft2) + movementPreference(move,
            board, playerId, size, recentPositions);
188     }
189
190     // применяет ход на копии доски и оценивает получившуюся позицию
191     private int scoreMoveAfterApply(Board board, Move move, int
        playerId, int size, int wallsLeft1, int wallsLeft2) {
192         Board copy = board.copy();
193         applyMove(copy, move);
194         int newWallsLeft1 = wallsLeft1;
195         int newWallsLeft2 = wallsLeft2;
196         if (move.getMoveType() == MoveType.PLACE_WALL) {
197             if (move.getPlayerId() == 1) {
198                 --newWallsLeft1;
199             } else {
200                 --newWallsLeft2;
201             }
202         }
203         return evaluate(copy, playerId, size, newWallsLeft1,
            newWallsLeft2);
204     }
205
206     private record SearchResult(Move move, int score) {}
207 }

```

## Реализация алгоритма AlphaBeta

Листинг Б.1 — Класс AlphaBetaAlgorithm

```

1 public class AlphaBetaAlgorithm implements Algorithm {
2     private final Map<String, TranspositionEntry>
        transpositionTable = new HashMap<>();
3     private final Map<String, Integer> goodMoves = new HashMap
        <>();
4     private final Move[][] bestMoves = new Move[50][2];
5     private AlgorithmReport lastReport;
6     private long nodesVisited;
7     private long consideredMoves;
8     private long cutoffs;
9     private long tableHits;
10    private List<Position> recentPositions = List.of();
11    private EvaluationWeights evaluationWeights;
12
13    @Override
14    public ComputerAlgorithmType getType() {
15        return ComputerAlgorithmType.ALPHABETA;
16    }
17
18    // хранит подробности последнего найденного хода.
19    @Override
20    public AlgorithmReport getLastReport() {
21        return lastReport;
22    }
23
24    // передает алгоритму историю последних позиций
25    @Override
26    public void setRecentPositions(List<Position>
        recentPositions) {
27        this.recentPositions = recentPositions == null ? List.
            of() : List.copyOf(recentPositions);
28    }
29
30    // позволяет использовать отдельные веса оценки

```

```

31     @Override
32     public EvaluationWeights getEvaluationWeights() {
33         return evaluationWeights == null ? Algorithm.super.
            getEvaluationWeights() : evaluationWeights;
34     }
35
36     // сохраняет веса, которые будут применяться именно этим экземпляром
    алгоритма
37     @Override
38     public void setEvaluationWeights(EvaluationWeights
        evaluationWeights) {
39         this.evaluationWeights = evaluationWeights;
40     }
41
42     // запускает поиск с постепенным увеличением глубины
43     @Override
44     public Move getMove(Board board,
        ComputerPlayerHardnessLevel level, int playerId, int
        wallsLeft1, int wallsLeft2) {
45         long startedAt = System.currentTimeMillis();
46         int size = (int) Math.sqrt(board.getTiles().size());
47         int maxDepth = getMaxDepth(level, size);
48         long timeLimit = getTimeLimit(level, size);
49         long endTime = System.currentTimeMillis() + timeLimit;
50
51         transpositionTable.clear();
52         nodesVisited = 0;
53         consideredMoves = 0;
54         cutoffs = 0;
55         tableHits = 0;
56
57         int currentWalls = playerId == 1 ? wallsLeft1 :
            wallsLeft2;
58         Move tacticalMove = findEndgameMove(board, playerId,
            currentWalls);
59         if (tacticalMove != null) {
60             int tacticalScore = scoreMoveAfterApply(board,
                tacticalMove, playerId, size, wallsLeft1,
                wallsLeft2);
61             lastReport = new AlgorithmReport(getType()).

```

```

        getDescription(), tacticalMove, tacticalScore,
        0, nodesVisited, 1, cutoffs, tableHits, System.
        currentTimeMillis() - startedAt, "AlphaBeta_прим
        енил_эндшпильное_правило:_немедленная_победа_или
        _срочная_блокировка");
62         return tacticalMove;
63     }
64
65     Move bestMove = null;
66     int bestScore = 0;
67     int reachedDepth = 0;
68     for (int depth = 1; depth <= maxDepth; ++depth) {
69         if (System.currentTimeMillis() > endTime) {
70             break;
71         }
72         SearchResult result = search(board, depth, playerId
            , size, wallsLeft1, wallsLeft2, endTime);
73         if (result.move() != null) {
74             bestMove = result.move();
75             bestScore = result.score();
76             reachedDepth = depth;
77         }
78     }
79     lastReport = new AlgorithmReport(getType().
        getDescription(), bestMove, bestScore, reachedDepth,
        nodesVisited, consideredMoves, cutoffs, tableHits,
        System.currentTimeMillis() - startedAt, "AlphaBeta_и
        спользует_сортировку_ходов,_beta_<=_alpha_отсечения_
        и_transposition_table_c_depth-bound_flags;_попаданий
        _в_таблицу:_"+ tableHits + ";_лимит_времени:_"+
        timeLimit + "_мс");
80     return bestMove;
81 }
82
83 // задает глубину поиска в зависимости от сложности и размера игрового поля
84 private int getMaxDepth(ComputerPlayerHardnessLevel level,
    int size) {
85     boolean smallBoard = size <= GameSize.SMALL.
        getAmountOfTilesPerSide();
86     boolean largeBoard = size > GameSize.NORMAL.

```

```

        getAmountOfTilesPerSide();
87     return switch (level) {
88         case EASY -> smallBoard ? 5 : 4;
89         case MEDIUM -> smallBoard ? 7 : 6;
90         case HARD -> largeBoard ? 7 : 8;
91     };
92 }
93
94 // задает лимит времени на ход
95 @Override
96 public long getTimeLimit(ComputerPlayerHardnessLevel level,
    int size) {
97     boolean largeBoard = size > GameSize.NORMAL.
        getAmountOfTilesPerSide();
98     boolean smallBoard = size <= GameSize.SMALL.
        getAmountOfTilesPerSide();
99     return switch (level) {
100         case EASY -> smallBoard ? 1000L : 1300L;
101         case MEDIUM -> largeBoard ? 4800L : 3600L;
102         case HARD -> largeBoard ? 9500L : 7600L;
103     };
104 }
105
106 // проверяет корневые ходы и выбирает лучший результат среди них
107 private SearchResult search(Board board, int depth, int
    playerId, int size, int wallsLeft1, int wallsLeft2, long
    endTime) {
108     int currentWalls = playerId == 1 ? wallsLeft1 :
        wallsLeft2;
109     List<Move> moves = new ArrayList<>(getMoves(board,
        playerId, currentWalls).stream()
110         .filter(move -> isRootMoveLegal(board, move,
            playerId, currentWalls)).toList());
111     orderMoves(moves, board, playerId, size, wallsLeft1,
        wallsLeft2, 0, null);
112     consideredMoves += moves.size();
113
114     Move bestMove = null;
115     int best = Integer.MIN_VALUE;
116     for (Move move : moves) {

```

```

117         if (System.currentTimeMillis() > endTime) {
118             break;
119         }
120         Board copy = board.copy();
121         applyMove(copy, move);
122
123         int newWallsLeft1 = wallsLeft1, newWallsLeft2 =
            wallsLeft2;
124         if (move.getMoveType() == MoveType.PLACE_WALL) {
125             if (playerId == 1) {
126                 --newWallsLeft1;
127             } else {
128                 --newWallsLeft2;
129             }
130         }
131         int val = alphaBeta(copy, depth - 1, Integer.
            MIN_VALUE, Integer.MAX_VALUE, false, playerId,
            size, newWallsLeft1, newWallsLeft2, endTime, 1);
132         val += rootMovePreference(move, board, playerId,
            size);
133         if (val > best) {
134             best = val;
135             bestMove = move;
136         }
137     }
138     return new SearchResult(bestMove, best);
139 }
140
141 //рекурсивный alpha-beta поиск с отсечениями и таблицей транспозиций
142 private int alphaBeta(Board board, int depth, int alpha,
    int beta, boolean max, int playerId, int size, int
    wallsLeft1, int wallsLeft2, long endTime, int ply) {
143     nodesVisited++;
144     if (System.currentTimeMillis() > endTime) {
145         return evaluate(board, playerId, size, wallsLeft1,
            wallsLeft2);
146     }
147     String key = boardKey(board) + "/" + max + "/" +
        wallsLeft1 + "/" + wallsLeft2;
148     int originalAlpha = alpha;

```

```

149         int originalBeta = beta;
150
151         TranspositionEntry cached = transpositionTable.get(key)
            ;
152         if (cached != null && cached.depth >= depth) {
153             if (cached.bound == Bound.EXACT) {
154                 tableHits++;
155                 return cached.score;
156             }
157             if (cached.bound == Bound.LOWER) {
158                 alpha = Math.max(alpha, cached.score);
159             } else if (cached.bound == Bound.UPPER) {
160                 beta = Math.min(beta, cached.score);
161             }
162             if (alpha >= beta) {
163                 tableHits++;
164                 return cached.score;
165             }
166         }
167
168         Integer terminal = terminalScore(board, playerId, size,
            depth);
169         if (terminal != null) {
170             transpositionTable.put(key, new TranspositionEntry(
                depth, terminal, Bound.EXACT, null));
171             return terminal;
172         }
173
174         if (depth == 0) {
175             if (isNoisyPosition(board, playerId, size)) {
176                 int calmScore = quiescence(board, alpha, beta,
                    max, playerId, size, wallsLeft1, wallsLeft2,
                    endTime, 2);
177                 transpositionTable.put(key, new
                    TranspositionEntry(depth, calmScore, Bound.
                        EXACT, null));
178                 return calmScore;
179             }
180             int eval = evaluate(board, playerId, size,
                wallsLeft1, wallsLeft2);

```

```

181         transpositionTable.put(key, new TranspositionEntry(
182             depth, eval, Bound.EXACT, null));
183     }
184
185     int current = max ? playerId : 3 - playerId;
186     int walls = current == 1 ? wallsLeft1 : wallsLeft2;
187
188     List<Move> moves = getMoves(board, current, walls);
189     Move tableBestMove = cached == null ? null : cached.
190         bestMove;
191     orderMoves(moves, board, playerId, size, wallsLeft1,
192         wallsLeft2, ply, tableBestMove);
193     if (moves.size() > 36) {
194         moves = moves.subList(0, 36);
195     }
196     consideredMoves += moves.size();
197
198     int best = max ? Integer.MIN_VALUE : Integer.MAX_VALUE;
199     Move bestMove = null;
200
201     for (Move move : moves) {
202         Board copy = board.copy();
203         applyMove(copy, move);
204
205         int newWallsLeft1 = wallsLeft1, newWallsLeft2 =
206             wallsLeft2;
207         if (move.getMoveType() == MoveType.PLACE_WALL) {
208             if (current == 1) {
209                 --newWallsLeft1;
210             } else {
211                 --newWallsLeft2;
212             }
213         }
214
215         int val = alphaBeta(copy, depth - 1, alpha, beta, !
216             max, playerId, size, newWallsLeft1,
217             newWallsLeft2, endTime, ply + 1);
218
219         if (max) {

```



```

215         if (val > best) {
216             best = val;
217             bestMove = move;
218         }
219         alpha = Math.max(alpha, val);
220     } else {
221         if (val < best) {
222             best = val;
223             bestMove = move;
224         }
225         beta = Math.min(beta, val);
226     }
227
228     if (beta <= alpha) {
229         cutoffs++;
230         updateBestMoves(move, ply);
231         updateGoodMoves(move);
232         break;
233     }
234 }
235
236 Bound bound = Bound.EXACT;
237 if (best <= originalAlpha) {
238     bound = Bound.UPPER;
239 } else if (best >= originalBeta) {
240     bound = Bound.LOWER;
241 }
242 transpositionTable.put(key, new TranspositionEntry(
243     depth, best, bound, bestMove));
244 return best;
245 }
246
247 //досматривает "острые"позиции около победы
248 private int quiescence(Board board, int alpha, int beta,
249     boolean max, int playerId, int size, int wallsLeft1, int
250     wallsLeft2, long endTime, int extensionDepth) {
    nodesVisited++;
    int standPat = evaluate(board, playerId, size,
        wallsLeft1, wallsLeft2);
    if (extensionDepth == 0 || System.currentTimeMillis() >

```

```

        endTime) {
251         return standPat;
252     }
253
254     if (max) {
255         if (standPat >= beta) {
256             return beta;
257         }
258         alpha = Math.max(alpha, standPat);
259     } else {
260         if (standPat <= alpha) {
261             return alpha;
262         }
263         beta = Math.min(beta, standPat);
264     }
265
266     int current = max ? playerId : 3 - playerId;
267     int walls = current == 1 ? wallsLeft1 : wallsLeft2;
268     List<Move> moves = getQuiescenceMoves(board, current,
        walls, playerId, size);
269     if (moves.isEmpty()) {
270         return standPat;
271     }
272     orderMoves(moves, board, playerId, size, wallsLeft1,
        wallsLeft2, 0, null);
273
274     for (Move move : moves) {
275         if (System.currentTimeMillis() > endTime) {
276             break;
277         }
278         Board copy = board.copy();
279         applyMove(copy, move);
280
281         int newWallsLeft1 = wallsLeft1;
282         int newWallsLeft2 = wallsLeft2;
283         if (move.getMoveType() == MoveType.PLACE_WALL) {
284             if (current == 1) {
285                 --newWallsLeft1;
286             } else {
287                 --newWallsLeft2;

```

```

288         }
289     }
290
291     int value = quiescence(copy, alpha, beta, !max,
        playerId, size, newWallsLeft1, newWallsLeft2,
        endTime, extensionDepth - 1);
292     if (max) {
293         if (value >= beta) {
294             cutoffs++;
295             return beta;
296         }
297         alpha = Math.max(alpha, value);
298     } else {
299         if (value <= alpha) {
300             cutoffs++;
301             return alpha;
302         }
303         beta = Math.min(beta, value);
304     }
305 }
306 return max ? alpha : beta;
307 }
308
309     // ставит в начало списка ходы, которые с большей вероятностью дадут
        хорошее отсечение
310     private void orderMoves(List<Move> moves, Board board, int
        playerId, int size, int wallsLeft1, int wallsLeft2, int
        ply, Move tableBestMove) {
311         moves.sort((a, b) -> Integer.compare(scoreMove(b, board
            , playerId, size, wallsLeft1, wallsLeft2, ply,
            tableBestMove), scoreMove(a, board, playerId, size,
            wallsLeft1, wallsLeft2, ply, tableBestMove)));
312     }
313
314     // считает эвристический балл хода для сортировки перед рекурсивным
        обходом
315     private int scoreMove(Move move, Board board, int playerId,
        int size, int wallsLeft1, int wallsLeft2, int ply, Move
        tableBestMove) {
316         int score = 0;

```

```

317
318     if (move.equals(tableBestMove)) {
319         score += 20000;
320     }
321
322     if (ply < bestMoves.length) {
323         if (bestMoves[ply][0] != null && move.equals(
324             bestMoves[ply][0])) {
325             score += 10000;
326         }
327         if (bestMoves[ply][1] != null && move.equals(
328             bestMoves[ply][1])) {
329             score += 8000;
330         }
331     }
332
333     score += goodMoves.getOrDefault(move.toString(), 0) *
334         2;
335     score += rootMovePreference(move, board, playerId, size
336         );
337     score += scoreMoveAfterApply(board, move, playerId,
338         size, wallsLeft1, wallsLeft2);
339
340     return score;
341 }
342
343 //быстро оценивает позицию, которая получится после выбранного хода
344 private int scoreMoveAfterApply(Board board, Move move, int
345     playerId, int size, int wallsLeft1, int wallsLeft2) {
346     Board copy = board.copy();
347     applyMove(copy, move);
348     int newWallsLeft1 = wallsLeft1;
349     int newWallsLeft2 = wallsLeft2;
350     if (move.getMoveType() == MoveType.PLACE_WALL) {
351         if (move.getPlayerId() == 1) {
352             --newWallsLeft1;
353         } else {
354             --newWallsLeft2;
355         }
356     }
357 }

```

```

351         return evaluate(copy, playerId, size, newWallsLeft1,
352             newWallsLeft2);
353     }
354     // добавляет небольшую поправку за движение к цели и против повторения
355     позиций
356     private int rootMovePreference(Move move, Board board, int
357         playerId, int size) {
358         return movementPreference(move, board, playerId, size,
359             recentPositions);
360     }
361     // проверяет, что ход из корня дерева действительно можно выполнить в
362     текущей позиции
363     private boolean isRootMoveLegal(Board board, Move move, int
364         playerId, int wallsLeft) {
365         if (move.getPlayerId() != playerId) {
366             return false;
367         }
368         if (move.getMoveType() == MoveType.MOVE_PLAYER) {
369             return board.getAvailableMoves(getCurrentPosition(
370                 board, playerId)).contains(move.getEndPosition()
371                 );
372         }
373         if (wallsLeft <= 0) {
374             return false;
375         }
376         return isWallPlaceable(board, move.getStartPosition(),
377             move.getEndPosition()) && isValidWall(board, move.
378             getStartPosition(), move.getEndPosition());
379     }
380 }
381
382 // запоминает ходы, на которых уже происходили отсечения на этой глубине
383 private void updateBestMoves(Move move, int ply) {
384     if (ply >= bestMoves.length || move.equals(bestMoves[
385         ply][0])) {
386         return;
387     }
388     bestMoves[ply][1] = bestMoves[ply][0];
389     bestMoves[ply][0] = move;

```

```

380     }
381
382     // повышает приоритет хода, если раньше он часто давал полезные отсечения
383     private void updateGoodMoves(Move move) {
384         goodMoves.merge(move.toString(), 1, Integer::sum);
385     }
386
387     private record SearchResult(Move move, int score) {}
388
389     private enum Bound {
390         EXACT,
391         LOWER,
392         UPPER
393     }
394
395     private record TranspositionEntry(int depth, int score,
396         Bound bound, Move bestMove) {}

```

## Реализация алгоритма Monte Carlo

Листинг В.1 — Класс MonteCarloAlgorithm

```

1 public class MonteCarloAlgorithm implements Algorithm {
2     private static final double C = 1.2;
3     private static final Random random = new Random();
4     private AlgorithmReport lastReport;
5     private List<Position> recentPositions = List.of();
6     private int rootPlayerId;
7
8     @Override
9     public ComputerAlgorithmType getType() {
10         return ComputerAlgorithmType.MONTECARLO;
11     }
12
13     //отдает статистику последнего запуска MCTS
14     @Override
15     public AlgorithmReport getLastReport() {
16         return lastReport;
17     }
18
19     //получает последние позиции игрока
20     @Override
21     public void setRecentPositions(List<Position>
22         recentPositions) {
23         this.recentPositions = recentPositions == null ? List.
24             of() : List.copyOf(recentPositions);
25     }
26
27     //строит дерево Монте-Карло и выбирает самый надежный дочерний ход
28     @Override
29     public Move getMove(Board board,
30         ComputerPlayerHardnessLevel hardnessLevel, int playerId,
31         int wallsLeft1, int wallsLeft2) {
32         long startedAt = System.currentTimeMillis();
33         int size = (int) Math.sqrt(board.getTiles().size());
34         long timeLimit = getTimeLimit(hardnessLevel, size);

```

```

31     long endTime = System.currentTimeMillis() + timeLimit;
32     int rolloutDepth = getRolloutDepth(hardnessLevel, size)
        ;
33     long iterationBudget = getIterationBudget(hardnessLevel
        , size);
34
35     long iterations = 0;
36     rootPlayerId = playerId;
37     int currentWalls = playerId == 1 ? wallsLeft1 :
        wallsLeft2;
38     Move tacticalMove = findEndgameMove(board, playerId,
        currentWalls);
39     if (tacticalMove != null) {
40         lastReport = new AlgorithmReport(getType().
            getDescription(), tacticalMove, scoreMove(board,
                tacticalMove, playerId, wallsLeft1, wallsLeft2)
                , 0, 0, 1, 0, 0, System.currentTimeMillis() -
                startedAt, "MCTS_применил_эндшпильное_правило_до
                _запуска_симуляций");
41         return tacticalMove;
42     }
43
44     Node root = new Node(board.copy(), null, null, playerId
        , wallsLeft1, wallsLeft2);
45     while (System.currentTimeMillis() < endTime &&
        iterations < iterationBudget) {
46         Node node = select(root);
47         Node expanded = expand(node);
48         int winner = simulate(expanded, rolloutDepth,
            playerId, size);
49         backpropagate(expanded, winner, playerId);
50         iterations++;
51         if (root.visits > 300 && bestWinRate(root) > 0.97)
            {
52             break;
53         }
54     }
55
56     Node best = root.children.stream()
57         .max(Comparator.comparingDouble(this::

```



```

        robustChildScore))
58         .orElseThrow();
59     lastReport = new AlgorithmReport(getType().
        getDescription(), best.move, evaluate(best.board,
        playerId, size, best.walls1, best.walls2),
        rolloutDepth, iterations, root.children.size(), 0,
        0, System.currentTimeMillis() - startedAt, "MCTS:_вы
        бор_по_UCT,_rollout_с_epsilon-greedy_эвристикой,_нем
        едленными_победами_и_тактическими_стенами;_лимит_вре
        мени:_" + timeLimit + "_мс,_бюджет_итераций:_" +
        iterationBudget);
60     return best.move;
61 }
62
63 //определяет длину одной случайной симуляции
64 private int getRolloutDepth(ComputerPlayerHardnessLevel
    level, int size) {
65     return switch (level) {
66         case EASY -> size * 4;
67         case MEDIUM -> size * 7;
68         case HARD -> size * 10;
69     };
70 }
71
72 //ограничивает количество симуляций для выбранной сложности
73 private long getIterationBudget(ComputerPlayerHardnessLevel
    level, int size) {
74     boolean largeBoard = size > GameSize.NORMAL.
        getAmountOfTilesPerSide();
75     return switch (level) {
76         case EASY -> largeBoard ? 380L : 520L;
77         case MEDIUM -> largeBoard ? 1300L : 1700L;
78         case HARD -> largeBoard ? 2600L : 3400L;
79     };
80 }
81
82 //задает временной лимит, после которого MCTS обязан вернуть лучший
    найденный ход
83 @Override
84 public long getTimeLimit(ComputerPlayerHardnessLevel level,

```

```

    int size) {
85     boolean largeBoard = size > GameSize.NORMAL.
        getAmountOfTilesPerSide();
86     return switch (level) {
87         case EASY -> largeBoard ? 1200L : 950L;
88         case MEDIUM -> largeBoard ? 3800L : 2900L;
89         case HARD -> largeBoard ? 7600L : 6200L;
90     };
91 }
92
93 // спускается по дереву в наиболее перспективный уже раскрытый узел
94 private Node select(Node node) {
95     while (!node.children.isEmpty() && node.untriedMoves.
        isEmpty()) {
96         node = node.children.stream().max(Comparator.
            comparing(this::uct)).orElseThrow();
97     }
98     return node;
99 }
100
101 // добавляет в дерево один новый ход из списка еще не проверенных
102 private Node expand(Node node) {
103     if (node.untriedMoves.isEmpty() && !node.children.
        isEmpty()) {
104         return node;
105     }
106
107     if (node.untriedMoves.isEmpty()) {
108         node.untriedMoves.addAll(getMoves(node.board, node.
            player, node.getWalls()));
109         node.untriedMoves.sort((a, b) -> Integer.compare(
110             scoreMove(node.board, b, node.player, node.
                walls1, node.walls2),
111             scoreMove(node.board, a, node.player, node.
                walls1, node.walls2)
112         ));
113     }
114
115     if (node.untriedMoves.isEmpty()) {
116         return node;

```

```

117     }
118
119     Move move = node.untriedMoves.remove(0);
120     Board copy = node.board.copy();
121     applyMove(copy, move);
122
123     int wallsLeft1 = node.walls1;
124     int wallsLeft2 = node.walls2;
125
126     if (move.getMoveType() == MoveType.PLACE_WALL) {
127         if (node.player == 1) {
128             --wallsLeft1;
129         } else {
130             --wallsLeft2;
131         }
132     }
133     Node child = new Node(copy, node, move, 3 - node.player
134         , wallsLeft1, wallsLeft2);
134     node.children.add(child);
135     return child;
136 }
137
138     //проигрывает партию вперед упрощенной стратегией и возвращает
139     предполагаемого победителя
139     private int simulate(Node node, int maxSteps, int
140         rootPlayer, int size) {
140         Board board = node.board.copy();
141         int currentPlayer = node.player;
142         int wallsLeft1 = node.walls1;
143         int wallsLeft2 = node.walls2;
144
145         for (int step = 0; step < maxSteps; step++) {
146             if (isWin(board, 1, size)) {
147                 return 1;
148             }
149             if (isWin(board, 2, size)) {
150                 return 2;
151             }
152
153             int walls = currentPlayer == 1 ? wallsLeft1 :

```

```

        wallsLeft2;
154     List<Move> moves = getMoves(board, currentPlayer,
        walls);
155     if (moves.isEmpty()) {
156         return 3 - currentPlayer;
157     }
158
159     Move best = pickRolloutMove(board, moves,
        currentPlayer, rootPlayer, size, wallsLeft1,
        wallsLeft2);
160
161     applyMove(board, best);
162
163     if (best.getMoveType() == MoveType.PLACE_WALL) {
164         if (currentPlayer == 1) {
165             --wallsLeft1;
166         } else {
167             --wallsLeft2;
168         }
169     }
170     currentPlayer = 3 - currentPlayer;
171 }
172
173     return evaluate(board, rootPlayer, size, wallsLeft1,
        wallsLeft2) > 0 ? rootPlayer : (3 - rootPlayer);
174 }
175
176     //поднимает результат симуляции вверх по дереву и обновляет счетчики
посещений
177     private void backpropagate(Node node, int winner, int
        playerId) {
178         while (node != null) {
179             ++node.visits;
180             if (winner == playerId) {
181                 ++node.wins;
182             }
183             node = node.parent;
184         }
185     }
186

```

```

187     // формула UCT
188     private double uct(Node n) {
189         if (n.visits == 0) {
190             return Double.MAX_VALUE;
191         }
192
193         double exploitation = n.wins / n.visits;
194         double exploration = C * Math.sqrt(Math.log(n.parent.
            visits) / n.visits);
195         double heuristic = normalizedEvaluate(n.board, n.parent
            .player, n.walls1, n.walls2);
196
197         return exploitation + exploration + heuristic;
198     }
199
200     // выбирает ход внутри rollout
201     private Move pickRolloutMove(Board board, List<Move> moves,
        int currentPlayer, int rootPlayer, int size, int
        wallsLeft1, int wallsLeft2) {
202         for (Move move : moves) {
203             if (move.getMoveType() == MoveType.MOVE_PLAYER
204                 && move.getEndPosition().row() ==
                getTargetRow(currentPlayer, size)) {
205                 return move;
206             }
207         }
208
209         int walls = currentPlayer == 1 ? wallsLeft1 :
            wallsLeft2;
210         if (walls > 0 && calculateShortestPath(board,
            getCurrentPosition(board, 3 - currentPlayer),
            getTargetRow(3 - currentPlayer, size)) <= 2) {
211             Move tacticalWall = moves.stream()
212                 .filter(move -> move.getMoveType() ==
                    MoveType.PLACE_WALL)
213                 .max(Comparator.comparingInt(move ->
                    wallImpact(board, move, currentPlayer,
                    size)))
214                 .orElse(null);
215             if (tacticalWall != null && wallImpact(board,

```

```

        tacticalWall, currentPlayer, size) > 0) {
217         return tacticalWall;
218     }
219 }
220
221 if (random.nextDouble() < 0.22) {
222     return moves.get(random.nextInt(moves.size()));
223 }
224
225 int sampleSize = Math.min(18, moves.size());
226 List<Move> sampled = new ArrayList<>(sampleSize);
227 Set<Integer> usedIndexes = new HashSet<>();
228 while (sampled.size() < sampleSize) {
229     int index = random.nextInt(moves.size());
230     if (usedIndexes.add(index)) {
231         sampled.add(moves.get(index));
232     }
233 }
234
235 sampled.sort((a, b) -> Integer.compare(
236     scoreRolloutMove(board, b, currentPlayer,
237         rootPlayer, size, wallsLeft1, wallsLeft2),
238     scoreRolloutMove(board, a, currentPlayer,
239         rootPlayer, size, wallsLeft1, wallsLeft2)
240 ));
241 int top = Math.max(1, Math.min(4, sampled.size()));
242 return sampled.get(random.nextInt(top));
243 }
244
245 // оценивает ход в симуляции с учетом стороны, за которую сейчас идет
246 rollout
247 private int scoreRolloutMove(Board board, Move move, int
248     currentPlayer, int rootPlayer, int size, int wallsLeft1,
249     int wallsLeft2) {
250     Board tmp = board.copy();
251     applyMove(tmp, move);
252     int newWallsLeft1 = wallsLeft1;
253     int newWallsLeft2 = wallsLeft2;
254     if (move.getMoveType() == MoveType.PLACE_WALL) {
255         if (currentPlayer == 1) {

```

```

251         --newWallsLeft1;
252     } else {
253         --newWallsLeft2;
254     }
255 }
256
257     int score = evaluate(tmp, rootPlayer, size,
258         newWallsLeft1, newWallsLeft2);
259     if (currentPlayer != rootPlayer) {
260         score = -score;
261     }
262     if (currentPlayer == rootPlayer) {
263         score += movementPreference(move, board,
264             currentPlayer, size, recentPositions);
265     }
266     return score;
267 }
268
269 //быстро оценивает ход для сортировки при раскрытии узла дерева
270 private int scoreMove(Board board, Move move, int playerId,
271     int wallsLeft1, int wallsLeft2) {
272     Board copy = board.copy();
273     applyMove(copy, move);
274     int size = (int) Math.sqrt(copy.getTiles().size());
275     int newWallsLeft1 = wallsLeft1;
276     int newWallsLeft2 = wallsLeft2;
277     if (move.getMoveType() == MoveType.PLACE_WALL) {
278         if (move.getPlayerId() == 1) {
279             --newWallsLeft1;
280         } else {
281             --newWallsLeft2;
282         }
283     }
284     int preference = playerId == rootPlayerId ?
285         movementPreference(move, board, playerId, size,
286             recentPositions) : 0;
287     return evaluate(copy, playerId, size, newWallsLeft1,
288         newWallsLeft2) + preference;
289 }

```

```

285     // нормализует оценку позиции
286     private double normalizedEvaluate(Board board, int playerId
        , int wallsLeft1, int wallsLeft2) {
287         int size = (int) Math.sqrt(board.getTiles().size());
288         return Math.tanh(evaluate(board, playerId, size,
            wallsLeft1, wallsLeft2) / 500.0) * 0.15;
289     }
290
291     // выбирает дочерний узел не только по победам, но и по устойчивости
        статистики
292     private double robustChildScore(Node node) {
293         if (node.visits == 0) {
294             return 0;
295         }
296         return node.visits + node.wins / node.visits;
297     }
298
299     // проверяет, насколько уверенно лучший ход уже выигрывает в симуляциях.
300     private double bestWinRate(Node root) {
301         return root.children.stream().mapToDouble(n -> n.visits
            == 0 ? 0 : n.wins / n.visits).max().orElse(0);
302     }
303
304     // проверяет достижение целевой строки конкретным игроком
305     public boolean isWin(Board board, int playerId, int size) {
306         int row = getCurrentPosition(board, playerId).row();
307         return (playerId == 1 && row == 0) || (playerId == 2 &&
            row == size - 1);
308     }
309
310     static class Node {
311         Board board;
312         Node parent;
313         List<Node> children = new ArrayList<>();
314         List<Move> untriedMoves = new ArrayList<>();
315         Move move;
316
317         int visits = 0;
318         double wins = 0;
319

```



```

320     int player;
321     int walls1, walls2;
322
323     Node(Board board, Node parent, Move move, int player,
          int w1, int w2) {
324         this.board = board;
325         this.parent = parent;
326         this.move = move;
327         this.player = player;
328         this.walls1 = w1;
329         this.walls2 = w2;
330     }
331
332     int getWalls() {
333         return player == 1 ? walls1 : walls2;
334     }
335 }
336 }

```

## Реализация обучения весов функции оценки

Листинг Г.1 — Класс EvaluationWeightsStore

```

1 public final class EvaluationWeightsStore {
2     private static final Path FILE = Path.of("data", "
        evaluation-weights.properties");
3     private static final Path HISTORY_FILE = Path.of("data", "
        evaluation-weights-history.csv");
4     private static EvaluationWeights current;
5
6     private EvaluationWeightsStore() {}
7
8     //возвращает актуальные веса оценки
9     public static synchronized EvaluationWeights current() {
10         if (current == null) {
11             current = load();
12         }
13         return current;
14     }
15
16     //обновляет веса по итогам партии
17     public static synchronized EvaluationTrainingUpdate
        learnFromGame(List<EvaluationLearningSample> samples,
        int winnerId, String source) {
18         EvaluationWeights beforeWeights = current();
19         if (samples.isEmpty() || winnerId == 0) {
20             return new EvaluationTrainingUpdate(false, winnerId
                , samples.size(), new int[7], beforeWeights,
                beforeWeights);
21         }
22
23         int[] gradient = new int[7];
24         for (EvaluationLearningSample sample : samples) {
25             int reward = sample.playerId() == winnerId ? 1 :
                -1;
26             EvaluationFeatures delta = sample.delta();
27             gradient[0] += reward * delta.pathAdvantage();

```

```

28         gradient[1] += reward * delta.myMobility();
29         gradient[2] += reward * delta.enemyMobilityPenalty
30             ();
31         gradient[3] += reward * delta.wallAdvantage();
32         gradient[4] += reward * delta.progressAdvantage();
33         gradient[5] += reward * delta.myEndgame();
34         gradient[6] += reward * delta.enemyEndgamePenalty()
35             ;
36     }
37
38     if (isZero(gradient)) {
39         return new EvaluationTrainingUpdate(false, winnerId
40             , samples.size(), gradient, beforeWeights,
41             beforeWeights);
42     }
43
44     current = beforeWeights.adjustedByGameGradient(gradient
45         , 1, samples.size());
46     save(current);
47     appendHistory(source, winnerId, samples.size(),
48         gradient, beforeWeights, current);
49     return new EvaluationTrainingUpdate(true, winnerId,
50         samples.size(), gradient, beforeWeights, current);
51 }
52
53 //загружает веса из properties-файла
54 private static EvaluationWeights load() {
55     if (!Files.exists(FILE)) {
56         EvaluationWeights defaults = EvaluationWeights.
57             defaults();
58         save(defaults);
59         return defaults;
60     }
61
62     Properties properties = new Properties();
63     try (Reader reader = Files.newBufferedReader(FILE,
64         StandardCharsets.UTF_8)) {
65         properties.load(reader);
66         return new EvaluationWeights(
67             readInt(properties, "pathAdvantage",

```

```

        EvaluationWeights.defaults().
            pathAdvantage()),
59         readInt(properties, "myMobility",
            EvaluationWeights.defaults().myMobility
            ()),
60         readInt(properties, "enemyMobility",
            EvaluationWeights.defaults().
            enemyMobility()),
61         readInt(properties, "wallAdvantage",
            EvaluationWeights.defaults().
            wallAdvantage()),
62         readInt(properties, "progressAdvantage",
            EvaluationWeights.defaults().
            progressAdvantage()),
63         readInt(properties, "myEndgame",
            EvaluationWeights.defaults().myEndgame()
            ),
64         readInt(properties, "enemyEndgame",
            EvaluationWeights.defaults().
            enemyEndgame()),
65         readLong(properties, "samples", 0)
66     );
67     } catch (IOException | IllegalArgumentException e) {
68         EvaluationWeights defaults = EvaluationWeights.
            defaults();
69         save(defaults);
70         return defaults;
71     }
72 }
73
74 // сохраняет текущий набор весов
75 private static void save(EvaluationWeights weights) {
76     try {
77         Files.createDirectories(FILE.getParent());
78         Files.writeString(FILE, serialize(weights),
            StandardCharsets.UTF_8);
79     } catch (IOException ignored) {}
80 }
81
82 // преобразует веса в простой properties-формат

```

```

83     private static String serialize(EvaluationWeights weights)
84     {
85         return ""
86         pathAdvantage=%d
87         myMobility=%d
88         enemyMobility=%d
89         wallAdvantage=%d
90         progressAdvantage=%d
91         myEndgame=%d
92         enemyEndgame=%d
93         samples=%d
94         "".formatted(
95             weights.pathAdvantage(),
96             weights.myMobility(),
97             weights.enemyMobility(),
98             weights.wallAdvantage(),
99             weights.progressAdvantage(),
100            weights.myEndgame(),
101            weights.enemyEndgame(),
102            weights.samples()
103        );
104    }
105
106    //дописывает строку в CSV-историю
107    private static void appendHistory(String source, int
108        winnerId, int sampleCount, int[] gradient,
109        EvaluationWeights before, EvaluationWeights after) {
110        try {
111            Files.createDirectories(HISTORY_FILE.getParent());
112            if (!Files.exists(HISTORY_FILE)) {
113                Files.writeString(HISTORY_FILE, historyHeader()
114                    , StandardCharsets.UTF_8);
115            }
116            Files.writeString(HISTORY_FILE, historyRow(source,
117                winnerId, sampleCount, gradient, before, after),
118                StandardCharsets.UTF_8, StandardOpenOption.
119                    APPEND);
120        } catch (IOException ignored) {}
121    }

```

```

117     // формирует заголовок CSV-файла с историей изменения весов
118     private static String historyHeader() {
119         return "timestamp,source,winnerId,sampleCount,"
120             + "gradientPath,gradientMyMobility,
              gradientEnemyMobility,gradientWall,
              gradientProgress,gradientMyEndgame,
              gradientEnemyEndgame,"
121         + "beforePath,beforeMyMobility,
              beforeEnemyMobility,beforeWall,
              beforeProgress,beforeMyEndgame,
              beforeEnemyEndgame,beforeSamples,"
122         + "afterPath,afterMyMobility,afterEnemyMobility
              ,afterWall,afterProgress,afterMyEndgame,
              afterEnemyEndgame,afterSamples\n";
123     }
124
125     // собирает одну строку истории
126     private static String historyRow(String source, int
        winnerId, int sampleCount, int[] gradient,
        EvaluationWeights before, EvaluationWeights after) {
127         return String.join(",",
128             LocalDateTime.now().toString(),
129             escape(source),
130             Integer.toString(winnerId),
131             Integer.toString(sampleCount),
132             Integer.toString(gradient[0]),
133             Integer.toString(gradient[1]),
134             Integer.toString(gradient[2]),
135             Integer.toString(gradient[3]),
136             Integer.toString(gradient[4]),
137             Integer.toString(gradient[5]),
138             Integer.toString(gradient[6]),
139             Integer.toString(before.pathAdvantage()),
140             Integer.toString(before.myMobility()),
141             Integer.toString(before.enemyMobility()),
142             Integer.toString(before.wallAdvantage()),
143             Integer.toString(before.progressAdvantage()),
144             Integer.toString(before.myEndgame()),
145             Integer.toString(before.enemyEndgame()),
146             Long.toString(before.samples()),

```

```

147         Integer.toString(after.pathAdvantage()),
148         Integer.toString(after.myMobility()),
149         Integer.toString(after.enemyMobility()),
150         Integer.toString(after.wallAdvantage()),
151         Integer.toString(after.progressAdvantage()),
152         Integer.toString(after.myEndgame()),
153         Integer.toString(after.enemyEndgame()),
154         Long.toString(after.samples())
155     ) + "\n";
156 }
157
158 // экранирует текстовое поле для корректной записи в CSV
159 private static String escape(String value) {
160     String safe = value == null ? "" : value.replace("\\",
161         "\\");
162     return "\"" + safe + "\"";
163 }
164
165 private static int readInt(Properties properties, String
166     key, int fallback) {
167     return Integer.parseInt(properties.getProperty(key,
168         Integer.toString(fallback)));
169 }
170
171 private static long readLong(Properties properties, String
172     key, long fallback) {
173     return Long.parseLong(properties.getProperty(key, Long.
174         toString(fallback)));
175 }
176
177 // проверяет, есть ли в градиенте хоть одно ненулевое изменение.
178 private static boolean isZero(int[] values) {
179     for (int value : values) {
180         if (value != 0) {
181             return false;
182         }
183     }
184     return true;
185 }

```